Secure IP 设计文档

1. IP 简介

`Secure IP` 是当前工程中的统一安全处理子系统，围绕 AES 分组加解密、SHA-256 哈希计算以及两者的组合数据通路构建，涵盖算法核、数据格式适配、DMA 读写、寄存器配置、流程控制、padding 处理与顶层集成时序。本文档将 `aes_core`、 `sha256_core` 和 `secure_combo` 三个子系统的设计说明按工程学习顺序汇总到一起，便于从算法核逐步过渡到组合安全 IP 的整体实现。

2. AES Core 子系统

2.1 aes_core 顶层模块设计说明

1. 模块定位

`aes_core` 是 AES 子系统的顶层控制与数据通路整合模块，负责：

1. 统一对外接口（key 配置、模式配置、启动命令、数据输入输出）。
2. 管理密钥扩展过程（通过 `aes_key_mem` ）。
3. 调度加密核 `aes_encipher` 和解密核 `aes_decipher`。
4. 在 CBC 模式下处理首块/后续块的异或链。
5. 输出块结果和结果有效脉冲。

它本身不直接实现每轮 AES 变换细节，而是把 key/path/mode 协调成完整可用的块处理引擎。

2. 子模块关系

```
                      +----------------------------------------+
                      |                aes_core                |
                      |                                        |
cfg_key_i ---------->|                                        |
cfg_key_len_i ------>|          +------------------+          |
cmd_init_i --------->|--------->|   aes_key_mem    |--+       |
                      |          | (key expansion & |  |round  |
                      |          |  round key read) |  |key    |
                      |          +------------------+  |       |
                      |                                v       |
                      |                     +----------------+ |
                      |                     |  round key mux | |
                      |                     +----------------+ |
                      |                             |          |
                      |                             |          |
                      |  +----------------+         |          |
data_block_i ------->|->| ECB/CBC pre-XOR|---------+----+     |
```

```
cfg_iv_i ----------->|  | (enc path)     |              |      |
cmd_first_block_i -->|  +----------------+              |      |
                     |                                  |      |
op_encrypt_i --------|------------------------------+   |      |
cmd_next_i --------->|                              |   |      |
sts_key_ready_o ---->|                              |   |      |
                     |   +----------------------+   |   |      |
                     |   |     aes_encipher     |<--+   |      |
                     |   +----------------------+       |      |
                     |                                  |      |
                     |   +----------------------+       |      |
                     |   |     aes_decipher     |<------+      |
                     |   +----------------------+              |
                     |            |                            |
                     |            v                            |
                     |  +------------------------------+      |
                     |  | CBC post-XOR (dec path only) |      |
                     |  +------------------------------+      |
                     |            |                            |
                     |            v                            |
                     |      result_reg/data_valid_o            |
                     +-----------------------------------------+
```

说明：

1. `aes_sbox_lut` 由 `aes_core` 实例化，并在“密钥扩展阶段/加密阶段”复用。
2. `aes_inv_sbox_lut` 在 `aes_decipher` 内部实例化。

3. 顶层接口

```
                                    +----------------------------+
                                    |                            |
op_encrypt_i          -----------> |                            |
cfg_iv_i[127:0]       -----------> |                            |
cmd_init_i            -----------> |                            |
cmd_next_i            -----------> |                            |
cmd_first_block_i     -----------> |          aes_core          |  -----------> sts_key_ready
cfg_block_mode_i[2:0]-----------> |                            |  -----------> data_block_o[
clk_i                 -----------> |                            |  -----------> data_valid_o
rst_ni                -----------> |                            |
cfg_key_i[255:0]      -----------> |                            |
cfg_key_len_i[1:0]    -----------> |                            |
data_block_i[127:0]   -----------> |                            |
                                    |                            |
                                    +----------------------------+
```

1. 控制接口

- `cmd_init_i` ：1 拍脉冲，触发密钥扩展。
- `sts_key_ready_o` ：高电平表示密钥扩展完成，可接收 `cmd_next_i` 。
- `cmd_next_i` ：1 拍脉冲，触发一块 128-bit 的加/解密任务。

- `op_encrypt_i` ：1=加密，0=解密。
- `cmd_first_block_i` ：当前任务是否批数据首块（CBC 链需要）。

2. 配置接口

- `cfg_key_i[255:0]` ：AES key 输入。
- `cfg_key_len_i` ： `0` 表示 128 位， `2` 表示 256 位。
- `cfg_block_mode_i` ： `0=ECB` ， `1=CBC` 。
- `cfg_iv_i[127:0]` ：CBC 初始向量。

3. 数据接口

- `data_block_i[127:0]` ：输入块， `cmd_next_i` 启动后需保持稳定到核采样完成。
- `data_block_o[127:0]` ：输出块。
- `data_valid_o` ：输出有效脉冲（1 拍）。

4. 数据路径详细行为

4.1 加密路径

1. 若 `cfg_block_mode_i=ECB` ：

- `enc_block = data_block_i`

2. 若 `cfg_block_mode_i=CBC` ：

- 首块： `enc_block = data_block_i ^ cfg_iv_i`
- 后续块： `enc_block = data_block_i ^ enc_prev_cipher`

3. 当 `aes_encipher` 完成（ `enc_done` ）时：

- 结果直接作为 `data_block_o`
- 同时更新 `enc_prev_cipher = new_enc_block` （为下一个 CBC 块做链）

4.2 解密路径

1. `dec_block = data_block_i` （输入密文直接送解密核）。
2. `aes_decipher` 输出 `new_dec_block` 后：

- ECB：直接输出 `new_dec_block`
- CBC：
  - 首块： `new_dec_block ^ cfg_iv_i`
  - 后续块： `new_dec_block ^ dec_prev_cipher[255:128]`

3. `dec_prev_cipher` 用移位寄存器保存“前一块密文”，在 `dec_next` 时推进。

5. 关键状态寄存器

1. `init_state_reg` ：标记当前是否处于密钥扩展相关 S-Box 使用阶段。

2. `enc_prev_cipher` ：CBC 加密链寄存器。
3. `dec_prev_cipher[255:0]` ：解密前一块密文链（通过 256 位窗口保存新旧两块）。
4. `first_blk_dec_reg` ：在解密启动时锁存 `cmd_first_block_i` ，避免 done 时控制位漂移。
5. `result_reg/result_valid_reg` ：输出寄存器及输出有效脉冲。

6. S-Box 复用策略

`aes_core` 对正向 S-Box 做了复用：

1. 密钥扩展阶段：

- `aes_key_mem` 通过 `sbox_word` 请求 32 位 S-Box 变换。

2. 加密阶段：

- `aes_encipher` 提供 128 位状态 `enc_sbox` 做 SubBytes。

3. 复用选择：

- `init_state_reg=1` 时，`aes_sbox_lut` 输入为 `{96'd0, sbox_word}`（仅低 32 位有效）。
- 其他时刻输入为 `enc_sbox`（128 位）。

这样减少了独立 S-Box 资源。

7. ready/done 脉冲处理

`aes_core` 内部用边沿检测把子核 ready 高电平转换为“完成事件”：

1. `posedge_enc_ready` / `posedge_dec_ready`。
2. 在该事件当拍置 `result_valid_next=1` ，形成 1 拍输出有效。
3. 避免直接依赖 ready 电平导致多拍重复输出。

8. 时序使用约束（软件/上层必须遵守）

1. 必须先 `cmd_init_i` ，等待 `sts_key_ready_o=1`。
2. 仅在 `sts_key_ready_o=1` 时发 `cmd_next_i`。
3. CBC 模式必须正确标记 `cmd_first_block_i`。
4. 每块输出以 `data_valid_o` 为准采样。

9. 设计边界

1. 一次只处理一块，吞吐由上层调度（非深流水并行多块）。
2. 输入稳定窗口依赖上层协议保证。

2.2 aes_encipher 模块设计说明

1. 模块定位

`aes_encipher` 是 AES 单块加密执行核，处理一块 128-bit 数据，从 `enc_next` 启动到 `enc_ready` 完成。

它实现 AES 加密轮函数：

1. 初始 AddRoundKey
2. 多轮：SubBytes + ShiftRows + MixColumns + AddRoundKey
3. 最后一轮：SubBytes + ShiftRows + AddRoundKey（无 MixColumns）

2. 接口说明

```
                                  +----------------------------+
                                  |                            |
enc_next             ----------->  |                            |
key_len[1:0]         ----------->  |                            |
enc_round_key[127:0] ----------->  |                            |  -----------> enc_round_idx
new_sbox[127:0]      ----------->  |        aes_encipher        |  -----------> sbox[127:0]
enc_block[127:0]     ----------->  |                            |  -----------> new_enc_block
clk                  ----------->  |                            |  -----------> enc_ready
rst_n                ----------->  |                            |
                                  |                            |
                                  +----------------------------+
```

1. `enc_next` ：1 拍启动脉冲。
2. `key_len` ：决定总轮数（2'd0 -> 10 轮（AES-128）；2'd2 -> 14 轮（AES-256））。
3. `enc_round_idx` ：当前轮索引，输出给 key memory 选取 round key。
4. `enc_round_key` ：当前轮密钥输入。
5. `sbox/new_sbox` ：SubBytes 前后状态（128 位）。
6. `enc_block` ：输入明文块（预处理后）。
7. `new_enc_block` ：加密结果块。
8. `enc_ready` ：完成指示。

3. 状态机

1. `IDLE` ：等待 `enc_next` ，锁存输入块与总轮数。
2. `INIT` ：执行轮 0 的 AddRoundKey。
3. `MAIN` ：

- 若当前为最后轮： `ShiftRows(SubBytes)` 后 AddRoundKey，完成。
- 否则： `MixColumns(ShiftRows(SubBytes))` 后 AddRoundKey，轮计数 +1。

4. 轮函数细节

1. SubBytes：通过外部 `aes_sbox_lut` 完成， `sbox` 输入来自当前状态寄存器， `new_sbox` 为逐字节替换结果。

2. ShiftRows： `aes_shiftrows()` 函数按照标准映射重排 16 字节。

3. MixColumns： `aes_mixcolumns()` 对每列做矩阵乘法，内部通过 `mul2/mul3` （GF 运算）实现。

4. AddRoundKey： `state ^ round_key` ，按 `{b0..b15}` 一一对齐。

5. 时序特征

1. 输出 `new_enc_block` 是内部状态寄存器内容。
2. 完成当拍拉高 `enc_ready` ,在下一个 `enc_next` 到来时会拉低。

6. 设计边界

1. 一次只处理一个块。
2. 依赖外部提供正确轮密钥序列。
3. 输入 `enc_block` 需在启动时有效。

2.3 aes_decipher 模块设计说明

1. 模块定位

`aes_decipher` 是 AES 单块解密执行核，对应 `aes_encipher` 的逆过程。

核心流程：

1. 初始 AddRoundKey（从最后轮密钥开始）
2. 多轮：InvShiftRows + InvSubBytes + AddRoundKey + InvMixColumns
3. 末轮：InvShiftRows + InvSubBytes + AddRoundKey（无 InvMixColumns）

2. 接口说明

```
                                  +----------------------------+
                                  |                            |
dec_next              ----------> |                            |
key_len[1:0]          ----------> |                            |
dec_round_key[127:0] ----------> |                            |  ----------> dec_round_idx
dec_block[127:0]      ----------> |        aes_decipher        |  ----------> new_dec_block
clk                   ----------> |                            |  ----------> dec_ready
rst_n                 ----------> |                            |
                                  |                            |
                                  +----------------------------+
```

1. `dec_next` ：1 拍启动脉冲。
2. `key_len` ：决定总轮数 10/14。
3. `dec_round_idx` ：当前轮索引（倒序）。
4. `dec_round_key` ：对应轮密钥输入。
5. `dec_block` ：输入密文块。
6. `new_dec_block` ：输出明文块（CBC 后处理在 `aes_core` 顶层做）。
7. `dec_ready` ：完成指示。

3. 状态机

1. `IDLE` 收到 `dec_next` 后，装载输入块，并把 `dec_round_idx` 置为 `total_rounds`。
2. `INIT` 执行第一次 AddRoundKey，轮计数减 1。
3. `MAIN`：

- 若 `dec_round_idx==0`：执行末轮（无 InvMixColumns），完成。
- 否则执行常规逆轮并继续减计数。

4. 逆轮函数细节

1. InvShiftRows：`aes_inv_shiftrows()` 实现行逆移位（row1 右1，row2 右2，row3 右3）。
2. InvSubBytes：内部实例化 `aes_inv_sbox_lut`。
    - 输入 `inv_sbox = aes_inv_shiftrows(state)`
    - 输出 `new_inv_sbox`
3. InvMixColumns：`aes_inv_mixcolumns()` 使用 GF 乘法系数 `{0e,0b,0d,09}`。内部提供 `mul9/mul11/mul13/mul14`。
4. AddRoundKey：仍是 `state ^ round_key`。

5. 与顶层的分工

`aes_decipher` 只做“裸 AES 解密块”。

CBC 模式中的链式异或（与 IV 或前一块密文）不在本模块内完成，而在 `aes_core` 顶层完成。

6. 时序特征

1. 完成当拍 `dec_ready=1`，在下一个 `dec_next` 到来时会拉低。
2. 轮索引倒序递减，便于从 key memory 读取逆向轮密钥。

7. 设计边界

1. 单块处理，不做多块并行。
2. 依赖外部密钥扩展正确性。

2.4 aes_key_mem 模块设计说明

1. 模块定位

`aes_key_mem` 是 AES 密钥扩展与轮密钥存储模块，负责：

1. 接收 128/256 位初始 key。
2. 按 AES 标准执行 key expansion。
3. 将每一轮 128-bit round key 存入 `key_mem`。

4. 按 `round_idx` 提供当前轮密钥给加/解密核。

这是 `aes_core` 的“轮密钥中心”。

2. 输入输出语义

```
                                    +----------------------------+
                                    |                            |
key_in[255:0]        ----------->   |                            |
key_len[1:0]         ----------->   |                            |
key_gen              ----------->   |                            |  -----------> key_ready
round_idx[3:0]       ----------->   |         aes_key_mem        |  -----------> round_key[127
new_sbox_word[31:0]  ----------->   |                            |  -----------> sbox_word[31:
clk                  ----------->   |                            |
rst_n                ----------->   |                            |
                                    |                            |
                                    +----------------------------+
```

1. `key_gen` ：1 拍脉冲，启动密钥扩展。
2. `key_len` ： `0=128bit` ， `2=256bit` 。
3. `key_in[255:0]` ：原始 key（128 位模式用高 128 位）。拼接顺序为{w0,w1,w2,w3,w4,w5,w6,w7}
4. `key_ready` ：高电平，表示扩展完成。
5. `round_idx[3:0]` ：轮号索引。
6. `round_key[127:0]` ：对应轮密钥。
7. `sbox_word/new_sbox_word` ：与外部 S-Box 的 32 位接口（扩展过程中使用）。

3. 内部寄存器与存储体

1. `key_mem[0:14]` ：最多 15 组轮密钥（AES-256 需要 0..14）。
2. `prev_key0_reg` 、 `prev_key1_reg` ：保存“上一组 key”的两个 128 位窗口（尤其服务 AES-256 的 256 位分组递推）。
3. `round_ctr_reg` ：当前生成到第几轮。
4. `rcon_reg` ：Rcon GF 递推值。
5. `ready_reg` ：扩展完成标志。
6. `key_mem_ctrl_reg` ：扩展 FSM 状态。

4. FSM 工作流程

状态：

1. `CTRL_IDLE`
2. `CTRL_INIT`
3. `CTRL_GENERATE`
4. `CTRL_DONE`

流程：

1. `IDLE` 等待 `key_gen`。
2. `INIT` 做初始化（计数器/rcon/key 暂存基准）。
3. `GENERATE` 每拍生成一个 128-bit round key 并写入 `key_mem`。
4. 达到目标轮数后进入 `DONE`，置 `key_ready=1`，再回 `IDLE`。

5. 密钥扩展机制

AES 密钥扩展（Key Expansion）的目标是：由用户输入的初始密钥生成各轮加/解密所需的轮密钥（Round Key）。算法以 32 bit 字（Word）为基本单位，记扩展后的字序列为 `W[i]`。

设：

- `Nk`：初始密钥包含的 32 bit 字数
- `Nr`：AES 加/解密轮数

则密钥扩展后**总共得到** `4 × (Nr + 1)` 个字，这些字中**包含初始密钥本身对应的前 `Nk` 个字**，其余部分由密钥扩展算法递推生成。每 4 个字组成 1 组 128 bit 轮密钥，因此总共对应 `Nr + 1` 组轮密钥。

AES 密钥扩展的通用递推关系如下：

- 当 `i mod Nk = 0` 时：
  `W[i] = W[i-Nk] ⊕ SubWord(RotWord(W[i-1])) ⊕ Rcon[i/Nk]`
- 当为 AES-256 且 `i mod Nk = 4` 时：
  `W[i] = W[i-Nk] ⊕ SubWord(W[i-1])`
- 其他情况：
  `W[i] = W[i-Nk] ⊕ W[i-1]`

其中：

- `RotWord()`：将 32 bit 字循环左移 8 bit
- `SubWord()`：对 32 bit 字中的 4 个字节分别进行 S-Box 替换
- `Rcon[]`：轮常量，仅在 `i mod Nk = 0` 的情况下参与运算

对于 AES-128，`Nk = 4`，`Nr = 10`。初始密钥对应 `W[0] ~ W[3]`，后续扩展生成 `W[4] ~ W[43]` 共 40 个字，因此总计得到 44 个字，对应 11 组 128 bit 轮密钥。

对于 AES-256，`Nk = 8`，`Nr = 14`。初始密钥对应 `W[0] ~ W[7]`，后续扩展生成 `W[8] ~ W[59]` 共 52 个字，因此总计得到 60 个字，对应 15 组 128 bit 轮密钥。

本设计的 RTL 实现以 **128 bit 轮密钥** 为单位进行生成与存储，而非逐个 32 bit 字单独输出。AES-128 模式下，初始密钥作为第 0 组轮密钥，之后每周期生成 1 组新的 128 bit 轮密钥；AES-256 模式下，初始 256 bit 密钥可视为前两组 128 bit 轮密钥，之后每周期继续生成 1 组新的 128 bit 轮密钥。

需要说明的是，标准算法中的递推单位是 32 bit 字 `W[i]`，而 RTL 中按 128 bit 轮密钥分组实现，因此两者仅是表达粒度不同，本质上与 AES 标准密钥扩展过程一致。

6. AES-128 密钥扩展

AES-128 中，`Nk = 4`，初始密钥可表示为：

- `W[0]`
- `W[1]`
- `W[2]`
- `W[3]`

之后对任意 `i ≥ 4`：

- 当 `i mod 4 = 0` 时：
  `W[i] = W[i-4] ⊕ SubWord(RotWord(W[i-1])) ⊕ Rcon[i/4]`

- 当 `i mod 4 ≠ 0` 时：
  `W[i] = W[i-4] ⊕ W[i-1]`

因此，AES-128 中每扩展出 4 个新字，就形成下一组 128 bit 轮密钥。对应关系为：

- 第 0 组轮密钥：`{W[0], W[1], W[2], W[3]}`
- 第 1 组轮密钥：`{W[4], W[5], W[6], W[7]}`
- ...
- 第 10 组轮密钥：`{W[40], W[41], W[42], W[43]}`

在 RTL 实现中，AES-128 模式下可将上一组轮密钥记为 `{w0, w1, w2, w3}`，则下一组轮密钥 `{k0, k1, k2, k3}` 可写为：

- `k0 = w0 ⊕ SubWord(RotWord(w3)) ⊕ Rcon`
- `k1 = w1 ⊕ k0`
- `k2 = w2 ⊕ k1`
- `k3 = w3 ⊕ k2`

该实现方式与标准递推公式等价，只是将 4 个字的生成合并为一组 128 bit 轮密钥并行完成。

7. AES-256 密钥扩展

AES-256 中，`Nk = 8`，初始密钥可表示为：

- `W[0] ~ W[7]`

这 8 个字对应前两组 128 bit 轮密钥。之后对任意 `i ≥ 8`，密钥扩展需要区分三种情况：

- 当 `i mod 8 = 0` 时：
  `W[i] = W[i-8] ⊕ SubWord(RotWord(W[i-1])) ⊕ Rcon[i/8]`

- 当 `i mod 8 = 4` 时：
  `W[i] = W[i-8] ⊕ SubWord(W[i-1])`

- 其他情况：
  `W[i] = W[i-8] ⊕ W[i-1]`

其中，`i mod 8 = 0` 对应 AES-256 的第一类特殊变换，包含 `RotWord()`、`SubWord()` 和 `Rcon`；`i mod 8 = 4` 对应第二类特殊变换，仅进行 `SubWord()`，不执行 `RotWord()`，也不引入 `Rcon`。

因此，AES-256 的轮密钥对应关系为：

- 第 0 组轮密钥：`{W[0], W[1], W[2], W[3]}`
- 第 1 组轮密钥：`{W[4], W[5], W[6], W[7]}`
- 第 2 组轮密钥：`{W[8], W[9], W[10], W[11]}`
- ...
- 第 14 组轮密钥：`{W[56], W[57], W[58], W[59]}`

在 RTL 实现中，AES-256 通常以两个相邻的 128 bit 寄存器共同保存当前 256 bit 密钥窗口。前两个周期输出初始密钥对应的两组 128 bit 轮密钥，之后每周期再生成一组新的 128 bit 轮密钥。虽然实现上常以“当前输出第几组 128 bit 轮密钥”来组织控制逻辑，但其本质仍严格对应标准算法中按 32 bit 字 `W[i]` 的递推规则。

8. Rcon 逻辑

1. `rcon_reg` 通过 GF(2^8) 乘 2 递推：

- `xtime` 风格：左移并在溢出时异或 `8'h1b`。

2. 启动时先设置特殊值 `8'h8d`，使第一次递推得到 `8'h01`，与 AES 标准对齐。

9. round_key 读取路径

`round_key` 为组合读：

- `round_key = key_mem[round_idx]`

这允许加/解密核按各自轮计数直接索引，不需要额外握手。

10. 设计要点

1. 扩展与使用解耦：先扩展完成，再运行数据块。
2. 统一 128/256 两种 key 长度。
3. 外部 S-Box 接口简化了模块复用。

2.5 aes_sbox_lut 模块设计说明

1. 模块定位

`aes_sbox_lut` 是 AES 正向 S-Box 查找模块，对输入 128 位状态的每个字节进行独立查表替换。

2. 输入输出

```
                                +----------------------------+
                                |                            |
sboxw[127:0]      ----------->|        aes_sbox_lut        | -----------> new_sboxw[127:0]
                                |                            |
                                +----------------------------+
```

1. `sboxw[127:0]` ：16 字节输入。
2. `new_sboxw[127:0]` ：16 字节输出。

映射关系：

- 对每个 `i ∈ [0,15]` ，有 `new_sboxw[8*i +: 8] = SBOX[sboxw[8*i +: 8]]` 。
- 即对输入 sboxw 的 16 个字节分别进行一次正向 S-Box 替换，拼接得到输出new_sboxw。

3. 实现结构

1. 内部声明 `sbox[0:255]` 常量表。
2. `generate for` 复制 16 路字节查找逻辑。
3. 纯组合逻辑，无时钟寄存器。

4. 在系统中的用途

1. `aes_encipher` 的 SubBytes（整块 128 位）。
2. `aes_key_mem` 的 SubWord（32 位）通过 `aes_core` 做输入复用。

2.6 aes_inv_sbox_lut 模块设计说明

1. 模块定位

`aes_inv_sbox_lut` 是 AES 逆向 S-Box 查找模块，供解密路径执行 InvSubBytes。

2. 输入输出

```
                                +----------------------------+
                                |                            |
sboxw[127:0]      ----------->  |      aes_inv_sbox_lut      | -----------> new_sboxw[127:0]
                                |                            |
                                +----------------------------+
```

1. `sboxw[127:0]` ：16 字节输入。
2. `new_sboxw[127:0]` ：16 字节输出。

字节级映射： `new_sboxw[i] = INV_SBOX[sboxw[i]]` 。

3. 实现结构

1. `inv_sbox[0:255]` 常量查表。

2. `generate for` 16 路并行查找。
3. 纯组合逻辑。

4. 在系统中的用途

1. 仅供 `aes_decipher` 使用。
2. 输入通常是 InvShiftRows 之后的状态。

2.7 axis32_to_block128 模块设计说明

1. 模块定位

`axis32_to_block128` 用于把 32-bit AXIS 字流打包成 128-bit 块，并缓存到小 FIFO 中，供 AES 核按块读取。

2. 接口语义

```
                                  +----------------------------+
                                  |                            |
s_axis_tdata[31:0]   ----------->  |                            |
s_axis_tvalid        ----------->  |                            |  -----------> s_axis_tready
s_axis_tlast         ----------->  |                            |  -----------> m_block_data[
m_block_pop          ----------->  |     axis32_to_block128     |  -----------> m_block_valid
clk                  ----------->  |                            |  -----------> ibuf_full
rst_n                ----------->  |                            |  -----------> ibuf_empty
                                  |                            |
                                  +----------------------------+
```

输入（AXIS Slave）：

1. `s_axis_tdata[31:0]`
2. `s_axis_tvalid`
3. `s_axis_tready`
4. `s_axis_tlast`（当前实现未参与打包结束判定）

输出（块接口，peek+pop）：

1. `m_block_data[127:0]`：FIFO 头块数据（窥视）。
2. `m_block_valid`：头块有效。
3. `m_block_pop`：外部消费完成后打一拍弹出。

状态输出：

1. `ibuf_full`
2. `ibuf_empty`

3. 内部结构

1. 组包器（packer）

- 收 4 个 32 位 beat，拼成 1 个 128 位块。
- `beat_cnt` 记录当前在第几拍（0..3）。
- `assembling` 标识是否处于组包中。

2. 块 FIFO（DEPTH x 128）

- `mem[]` + `wr_ptr/rd_ptr/count`。
- 支持 push/pop 同拍更新计数。

4. 字节序处理

每拍先做 `w_swapped` ：`{byte0,byte1,byte2,byte3}` 方向重排

然后按顺序放入：

- beat0 -> `block[127:96]`
- beat1 -> `block[95:64]`
- beat2 -> `block[63:32]`
- beat3 -> `block[31:0]`

该规则与 AES 核字节序约定对齐。

为什么要这样做：

1. AXIS/内存侧看到的 `32-bit word` 字节顺序，与 AES 核内部理解 `128-bit block` 的字节顺序并不完全一致。
2. 因此在“4 个 32-bit beat -> 1 个 128-bit AES block”之前，必须先把每个 word 内部字节顺序调整到 AES 核期望的形式。
3. `w_swapped` 本质上不是算法运算，而是“AXIS word view -> AES block view”的格式适配。
4. 如果不做这一步，常见现象不是算法完全错误，而是每个 32-bit word 内部字节顺序颠倒，导致 AES 结果和标准向量不匹配。

5. ready/valid 策略

1. 组包未开始时：仅在 FIFO 未满时 `s_axis_tready=1`。
2. 组包开始后：`s_axis_tready` 保持 1，确保 4 拍组包连续完成。

设计目的：防止“收了前几拍但后几拍因 backpressure 卡住”造成块边界混乱。

6. 外部使用规范

1. 仅当 `m_block_valid=1` 时可读取 `m_block_data`。
2. 处理完一个块后，外部给 `m_block_pop=1` 一拍。
3. 支持流水：在 pop 的同时可继续 push 新块。

2.8 block128_to_axis32 模块设计说明

1. 模块定位

`block128_to_axis32` 把 AES 输出的 128-bit 块拆分成 4 个 32-bit AXIS beat，并提供输出缓冲。

2. 接口语义

```
                                  +----------------------------+
                                  |                            |
s_block_data[127:0]  -----------> |                            |
s_block_valid        -----------> |                            |  -----------> s_block_ready
m_axis_tready        -----------> |                            |  -----------> m_axis_tdata[
clk                  -----------> |     block128_to_axis32     |  -----------> m_axis_tvalid
rst_n                -----------> |                            |  -----------> m_axis_tlast
                                  |                            |  -----------> obuf_full
                                  |                            |  -----------> obuf_empty
                                  +----------------------------+
```

输入（块接口）：

1. `s_block_data[127:0]`
2. `s_block_valid`
3. `s_block_ready`

输出（AXIS Master）：

1. `m_axis_tdata[31:0]`
2. `m_axis_tvalid`
3. `m_axis_tready`
4. `m_axis_tlast`（第4拍拉高）

状态输出：

1. `obuf_full`
2. `obuf_empty`

3. 内部结构

1. 输入块 FIFO（DEPTH x 128）

- 接收上游块并缓存。
- `push_128 = s_block_valid & s_block_ready`。

2. 发送引擎

- `sending` 标识正在发送一个 128-bit 块。
- `beat_idx` 指示当前输出第几拍（0..3）。
- `out_reg` 持有当前要发送的整块。

4. 发送流程

1. 当 `sending=0` 且 FIFO 非空时，加载 `out_reg=mem[rd_ptr]` ，开始发送。
2. 每个 `axis_hs` 发送一个 32-bit word。
3. `beat_idx==3` 的握手拍输出 `tlast=1` 并 `pop_128` 。
4. 块发完后回到待加载状态。

5. 字节序处理

从 `out_reg` 取字顺序：

1. beat0: `[127:96]`
2. beat1: `[95:64]`
3. beat2: `[63:32]`
4. beat3: `[31:0]`

再对每个 32-bit word 做字节重排后输出 `m_axis_tdata` 。

为什么这里还要再重排一次：

1. `axis32_to_block128` 在入口处已经把 AXIS/内存侧格式转换成了 AES 内部 block 格式。
2. 因此 AES 核输出的 `128-bit block` 仍然保持的是“AES 内部字节顺序”。
3. `block128_to_axis32` 的任务就是做反向转换，也就是“AES block view -> AXIS word view”。
4. 如果这里不重排，AES 核本身可能已经算对，但写回内存后的每个 32-bit word 字节顺序会和软件预期不一致。
5. 因此这两个模块本质上是一对对称适配器：入口负责转进 AES 视角，出口负责转回 AXIS/内存视角。

6. backpressure 行为

1. `m_axis_tvalid` 在 `sending=1` 时保持有效。
2. 若下游 `m_axis_tready=0` ，当前 beat 保持不变，不丢拍。
3. 上游是否可继续送块由 `obuf_full` 决定。

7. 设计要点

1. 把块接口和 AXIS 接口彻底解耦。
2. 保证 4 拍边界严格对齐并正确产生 `tlast` 。
3. 与 `axis32_to_block128` 配对，形成 AES 核前后适配闭环。

3. SHA256 Core 子系统

3.1 sha256_core 顶层模块设计说明

1. 模块职责与边界

`sha256_core` 是 SHA-256 压缩核的顶层控制器，负责“单个 512-bit 块”的完整处理流程控制，并在多块消息场景下维护链值（ `H0..H7` ）。

本模块负责：

1. 接收块级命令（ `cmd_valid/cmd_ready/cmd_init/cmd_last` ）。
2. 接收 16 个 32-bit 数据字（AXIS 输入）。
3. 驱动 64 轮压缩（0..15 轮边收边算，16..63 轮内部运算）。
4. 在末块完成时输出 `digest_valid` 脉冲与 256-bit digest。

本模块不负责：

1. 消息 padding（由上游路径模块完成）。
2. 多块切分策略（由上游决定每块数据与 `cmd_*` 标志）。

2. 顶层结构

```
+-------------------------------------------------+
|                  sha256_core                    |
|                                                 |
|   cmd_valid/cmd_init/cmd_last                   |
|                  |                              |
|                  v                              |
|        +----------------------------+           |
|        |  Top FSM + control regs    |           |
|        |  - state_q                 |           |
|        |  - round_idx_q             |           |
|        |  - last_block_q            |           |
|        |  - cmd_ready / s_axis_ready|           |
|        +-------------+--------------+           |
|                      |                          |
|          +-----------+-----------+              |
|          |                       |              |
|          v                       v              |
|   +-------------+        +---------------+      |
|   | sha256_wreg |        | sha256_k_lut  |      |
|   |  W[t] gen   |        |   K[t] gen    |      |
|   +------+------+        +-------+-------+      |
|          |                       |              |
|          +-----------+-----------+              |
|                      v                          |
|               +--------------+                  |
|               | sha256_loop  |                  |
|               | round engine |                  |
|               +------+-------+                  |
|                      | work_out[255:0]          |
|                      v                          |
|        +-----------------------------+          |
|        | H_chain_q / digest registers |         |
|        | - chain accumulate           |         |
|        | - last-block digest output   |         |
```

```
|         +-----------------------------+          |
|                                                  |
+--------------------------------------------------+
```

3. 端口定义与协议

```
                                     +----------------------------+
                                     |                            |
cmd_valid_i          ----------->    |                            |
cmd_init_i           ----------->    |                            |
cmd_last_i           ----------->    |        sha256_core         |  -----------> cmd_ready_o
clk_i                ----------->    |                            |  -----------> s_axis_ready_
rst_ni               ----------->    |                            |  -----------> digest_o[255:
s_axis_data_i[31:0]  ----------->    |                            |  -----------> digest_valid_
s_axis_valid_i       ----------->    |                            |
s_axis_last_i        ----------->    |                            |
                                     |                            |
                                     +----------------------------+
```

3.1 命令接口（块级）

1. `cmd_valid_i` ：上游发起“处理 1 个块”的请求。
2. `cmd_ready_o` ：本核可接收命令（仅 `IDLE` 为 1）。
3. `cmd_init_i` ：首块标志。为 1 时本块从 IV 开始。
4. `cmd_last_i` ：末块标志。为 1 时本块完成后输出 digest。

命令握手： `cmd_fire = cmd_valid_i & cmd_ready_o` 。

注意：

- 上游只能在 `cmd_ready_o=1` 时发起新块。
- 每个 `cmd_fire` 必须对应后续完整 16 字输入。

3.2 数据接口（字级 AXIS）

1. `s_axis_data_i[31:0]` ：当前输入字。
2. `s_axis_valid_i` ：输入字有效。
3. `s_axis_ready_o` ：本核可接收输入字（在 `RECV` 状态有效）。
4. `s_axis_last_i` ：块内最后 1 个字标志（第 16 个字时置 1）。

数据握手： `s_fire = s_axis_valid_i & s_axis_ready_o` 。

注意：

- 一个块必须恰好 16 次 `s_fire` 。
- 第 16 次 `s_fire` 时 `s_axis_last_i` 应为 1。

3.3 digest 输出

1. `digest_o[255:0]` ：消息摘要输出。

2. `digest_valid_o` ：1 拍脉冲，表示 `digest_o` 有效。

注意：

- 仅当本块 `cmd_last_i=1` 时，才在完成后触发 `digest_valid_o`。
- 非末块只更新链值，不输出最终 digest。

4. 状态机逐状态说明

状态集合：`IDLE -> INIT -> RECV -> RUN -> FINAL -> DONE`。

4.1 IDLE

动作：

1. `cmd_ready_o=1` ，等待命令。
2. 命令握手后锁存 `cmd_last_i` 到 `last_block_q`。
3. `round_idx` 清零。
4. `cmd_init_i=1` 时 `H_chain` 装载标准 IV，否则保持现有链值。

转移：

1. `cmd_fire` 后进入 `INIT`。

4.2 INIT

动作：

1. 对 `sha256_loop` 发出 1 拍初始化（ `init_en=1` ）。
2. 让工作寄存器 `a..h` 装载 `H_chain_q`。

转移：

1. 下一拍进入 `RECV`。

4.3 RECV（轮 0..15）

动作：

1. `s_axis_ready_o=1` ，接收输入字。
2. 每次 `s_fire` ：

- 推进 `sha256_wreg` 窗口（ `step_en=1` ）。
- 同拍执行 `sha256_loop` 一轮。
- `round_idx++`。

转移：

1. 当接收到第 16 字（ `round_idx==15` 且 `s_axis_last_i=1` ）时，转 `RUN`。

4.4 RUN（轮 16..63）

动作：

1. 不再依赖 AXIS 输入，`step_en` 每拍为 1。
2. `sha256_wreg` 自行展开 `W[t]`。
3. `sha256_loop` 完成剩余轮计算。

转移：

1. 当 `round_idx==63` 后进入 `FINAL`。

4.5 FINAL

动作：

1. 执行链值累加：`H' = H_chain_q + work_out`（分 8 个 32-bit 相加）。
2. 更新 `H_chain_d = H'`。

转移：

1. `last_block_q=1` -> `DONE`。
2. 否则回到 `IDLE`，等待下一块命令。

4.6 DONE

动作：

1. `digest_q <= H_chain_q`。
2. `digest_valid_q <= 1`（仅 1 拍）。

转移：

1. 下一拍回 `IDLE`。

5. 关键内部寄存器

1. `state_q`：FSM 当前状态。
2. `round_idx_q[5:0]`：0..63 轮计数。
3. `H_chain_q[255:0]`：当前链值（H0..H7）。
4. `last_block_q`：当前块是否末块。
5. `digest_q/digest_valid_q`：digest 输出寄存与有效脉冲。
6. `s_axis_ready_q`：输入 ready 时序寄存。

6. 子模块协同关系

1. `sha256_k_lut`：按 `round_idx_q` 输出轮常数 `K[t]`。
2. `sha256_wreg`：根据 `round_idx` 和历史窗口输出 `W[t]`。

3. `sha256_loop` ：执行每轮 `a..h` 更新，输出 `work_out` 。

协同节拍：

1. `RECV` 阶段：每个 `s_fire` 同时推进 `wreg` 和 `loop` 。
2. `RUN` 阶段：无需输入字， `wreg` 和 `loop` 仍每拍推进。

7. 典型时序（单块）

1. `IDLE` ： `cmd_fire` 。
2. `INIT` ：装载 `a..h = H_chain` 。
3. `RECV` ：16 次 `s_fire` ，完成轮 0..15。
4. `RUN` ：48 拍，完成轮 16..63。
5. `FINAL` ：链值相加。
6. `DONE` ： `digest_valid_o` 拉高 1 拍（仅末块）。

8. 常见联调问题与定位

1. 现象：状态卡在 `RECV` 。

- 原因：第 16 字没有带 `s_axis_last_i=1` ，或输入不足 16 字。

2. 现象：digest 与软件不一致。

- 原因：上游 padding/分块错误，或 `cmd_init/cmd_last` 标志给错。

3. 现象：多块消息后 digest 错。

- 原因：中间块误把 `cmd_init_i` 置 1，导致链值被 IV 重置。

4. 现象：没有 digest_valid。

- 原因：末块 `cmd_last_i` 未置 1。

9. 设计结论

`sha256_core` 通过“块命令 + 16 字输入 + 64 轮压缩 + 末块输出”完成标准 SHA-256 压缩流程。其分层结构清晰：

1. 顶层 FSM 管控制与协议。
2. `wreg` 管消息调度。
3. `loop` 管轮函数。
4. `k_lut` 管常数。

因此可在不修改底层算法模块的前提下，稳定对接不同上游数据路径。

3.2 sha256_wreg 模块设计说明

1. 模块定位

`sha256_wreg` 是 SHA-256 的消息调度模块（Message Schedule），负责在每一轮给出 `W[t]`。

核心思想：

1. 用 16 个 32-bit 寄存器构成滑动窗口。
2. 前 16 轮直接使用输入字。
3. 后 48 轮按标准公式在线展开。

2. 接口定义

```
                              +----------------------------+
                              |                            |
clr_i            -----------> |                            |
step_en          -----------> |                            |
round_idx[5:0]   -----------> |        sha256_wreg         | -----------> Wt[31:0]
in_word[31:0]    -----------> |                            |
clk              -----------> |                            |
rst_n            -----------> |                            |
                              |                            |
                              +----------------------------+
```

1. `clk` / `rst_n`：时钟与低有效复位。
2. `clr_i`：清空窗口（通常在新块命令握手时拉高 1 拍）。
3. `step_en`：推进 1 轮（窗口左移并写入新 `Wt`）。
4. `round_idx[5:0]`：当前轮号。
5. `in_word[31:0]`：轮 0..15 的输入字。
6. `Wt[31:0]`：当前轮消息字输出。

3. 算法定义

当 `round_idx < 16`：`Wt = in_word`。

当 `round_idx >= 16`：`Wt = sigma1(W[t-2]) + W[t-7] + sigma0(W[t-15]) + W[t-16]`。

函数定义：

1. `sigma0(x) = ROTR7(x) ^ ROTR18(x) ^ SHR3(x)`
2. `sigma1(x) = ROTR17(x) ^ ROTR19(x) ^ SHR10(x)`

注：这里是小 sigma（消息展开函数），不是大 `SIGMA0/SIGMA1`。

4. 窗口组织与索引映射

内部数组：`wreg[0..15]`。

当前实现映射关系：

1. `wreg[0] -> W[t-16]`
2. `wreg[1] -> W[t-15]`
3. `wreg[9] -> W[t-7]`
4. `wreg[14] -> W[t-2]`

每次 `step_en=1` 的时序动作：

1. `wreg[0] <= wreg[1]`
2. `wreg[1] <= wreg[2]`
3. ...
4. `wreg[14] <= wreg[15]`
5. `wreg[15] <= wt_calc`

等价含义：

1. 窗口整体左移 1 位。
2. 新 `Wt` 入队尾。

5. 控制优先级

时序 always 块优先级：

1. `!rst_n` ：全部清零。
2. `clr_i` ：全部清零。
3. `step_en` ：执行一次窗口推进。
4. 其他：保持。

设计意图：复位和新块清理必须最高优先级，避免跨块污染。

6. 与顶层配合关系

1. `sha256_core` 在每次新块命令握手时触发 `clr_i` 。
2. `sha256_core` 在 `RECV` （收到字）和 `RUN` （内部轮）阶段给 `step_en` 。
3. `round_idx` 与 `step_en` 同步推进，保证 `Wt/Kt/loop` 对齐。

7. 边界条件

1. `round_idx==15 -> 16` 的切换点：下一拍立即从“直通输入字”切到“公式展开字”。
2. `step_en=0` 时：窗口不移动， `Wt` 仅按当前 `round_idx/in_word` 组合计算。
3. `clr_i` 和 `step_en` 同拍时：由于 `clr_i` 优先级更高，窗口清零，不做推进。

8. 设计总结

`sha256_wreg` 用“16 字窗口 + 在线展开”替代完整 64 字 RAM，资源低、时序清晰。只要顶层保证 `round_idx/step_en` 对齐，它就能稳定地产生标准 SHA-256 的 `W[t]` 序列。

3.3 sha256_loop 模块设计说明

1. 模块定位

`sha256_loop` 是 SHA-256 的轮函数执行单元。它维护工作寄存器 `a,b,c,d,e,f,g,h` ，并在每个 `step_en` 周期完成一轮状态更新。

该模块是算法运算核心，特点是：

1. 单拍一轮。
2. 输入 `Wt/Kt` 与当前 `a..h` ，输出下一轮 `a..h` 。
3. 支持由 `init_en` 重新装载链值。

2. 接口定义

```
                              +----------------------------+
                              |                            |
init_en          -----------> |                            |
step_en          -----------> |                            |
Wt_in[31:0]      -----------> |        sha256_loop         | -----------> work_out[255:0]
Kt_in[31:0]      -----------> |                            |
H_in[255:0]      -----------> |                            |
clk              -----------> |                            |
rst_n            -----------> |                            |
                              |                            |
                              +----------------------------+
```

1. `clk` / `rst_n` ：时钟与低有效复位。
2. `init_en` ：初始化工作寄存器（ `a..h <= H_in` ）。
3. `step_en` ：推进一轮。
4. `Wt_in[31:0]` ：当前轮消息字。
5. `Kt_in[31:0]` ：当前轮常数。
6. `H_in[255:0]` ：链值输入（拼接顺序 `{H0,H1,H2,H3,H4,H5,H6,H7}` ）。
7. `work_out[255:0]` ：当前工作寄存器输出（ `{a,b,c,d,e,f,g,h}` ）。

3. 数学公式

实现标准 SHA-256 轮函数：

1. `T1 = h + SIGMA1(e) + Ch(e,f,g) + Kt + Wt`
2. `T2 = SIGMA0(a) + Maj(a,b,c)`
3. `a' = T1 + T2`
4. `e' = d + T1`
5. 其余移位：

- `b' = a`
- `c' = b`

- `d' = c`
- `f' = e`
- `g' = f`
- `h' = g`

逻辑函数：

1. `SIGMA0(x)=ROTR2 ^ ROTR13 ^ ROTR22`
2. `SIGMA1(x)=ROTR6 ^ ROTR11 ^ ROTR25`
3. `Ch(x,y,z)=(x&y)^(~x&z)`
4. `Maj(x,y,z)=(x&y)^(x&z)^(y&z)`

4. 时序与优先级

寄存器更新优先级：

1. `!rst_n` ： `a..h` 清零。
2. `init_en` ： `a..h <= H_in` 。
3. `step_en` ：执行一轮更新。
4. 其他：保持不变。

关键点：

1. `init_en` 优先于 `step_en` ，防止初始化拍被误当作计算拍。
2. `work_out` 始终反映当前寄存器值。

5. 数据路径拆分

代码中将 `T1` 分解为多个中间量，例如：

1. `tmp_Ch_sumE = Ch(e,f,g) + SIGMA1(e)`
2. `tmp_K_W = Kt_in + Wt_in`
3. `tmp_H_K_W = h + tmp_K_W`
4. `t1 = tmp_H_K_W + tmp_Ch_sumE`

有助于综合后组合路径分解。

6. 与其他模块配合

1. `sha256_k_lut` 提供 `Kt_in` 。
2. `sha256_wreg` 提供 `Wt_in` 。
3. `sha256_core` 提供 `init_en/step_en` 控制并在块末使用 `work_out` 与链值相加。 `work_out` 是“当前轮完成后的寄存器状态”，顶层需在正确状态读取。

3.4 sha256_k_lut 模块设计说明

1. 模块定位

`sha256_k_lut` 提供 SHA-256 的 64 个轮常数 `K[0..63]` 。这是一个纯组合、无状态查找模块。

2. 接口定义

```
                                +----------------------------+
                                |                            |
round_idx[5:0]   -----------> |        sha256_k_lut        | -----------> k_out[31:0]
                                |                            |
                                +----------------------------+
```

1. `round_idx[5:0]` ：轮号输入，范围 0..63。
2. `k_out[31:0]` ：当前轮对应常数。

3. 实现方式

1. 使用组合 `case(round_idx)` 显式列出 64 个常量。
2. `default` 分支输出 `32'd0` 作为防御行为。
3. 与标准文本常量表一一对应，易审查。
4. 综合稳定，通常映射为组合逻辑/ROM 形式。

4. 功能语义

1. 当 `round_idx=t` 时，输出 `K[t]` 。
2. 输出与 `round_idx` 同拍变化，不需要时钟。

配合关系：

1. 顶层每推进一轮， `round_idx` 递增。
2. `sha256_loop` 同拍读取 `Kt_in` 参与 `T1` 计算。

5. 设计总结

`sha256_k_lut` 结构简单，只要常数表和索引映射正确，整个 SHA-256 轮计算的常数输入链路即可视为可靠。

4. Secure Combo 子系统

4.1 secure_cfg 模块设计说明

1. 模块定位

`secure_cfg` 是 `secure_combo` 的 APB 配置与状态寄存器模块，职责如下：

1. 接收 CPU 通过 APB 写入的任务配置（模式、源/目的地址、长度、AES Key/IV）。
2. 在写 `CTRL.start` 时产生单拍 `start_pulse_o` ，触发后级控制器启动任务。
3. 将后级状态 `busy_i/done_i/err_i/err_code_i/err_src_i/state_dbg_i` 映射到可读状态寄存器。

4. 将后级 `hash_i[0..7]` 映射到可读 HASH 寄存器窗口。

该模块本身不做 AES/SHA 计算，只负责“配置与可观测性”。

2. 接口说明

```
                                          +----------------------------+
                                          |                            |
clk_i                        ----------->  |                            |  -----------> apb
rst_ni                       ----------->  |                            |  -----------> apb
apb_psel_i                   ----------->  |                            |  -----------> sta
apb_penable_i                ----------->  |                            |  -----------> int
apb_pwrite_i                 ----------->  |                            |  -----------> mod
apb_paddr_i[11:0]            ----------->  |                            |  -----------> aes
apb_pwdata_i[31:0]           ----------->  |                            |  -----------> aes
busy_i                       ----------->  |                            |  -----------> aes
done_i                       ----------->  |         secure_cfg         |  -----------> tag
err_i                        ----------->  |                            |  -----------> src
err_code_i[3:0]              ----------->  |                            |  -----------> dst
err_src_i[3:0]               ----------->  |                            |  -----------> msg
state_dbg_i[7:0]             ----------->  |                            |  -----------> aes
hash_i[0:7][31:0]            ----------->  |                            |  -----------> aes
                                          |                            |
                                          +----------------------------+
```

1. 时钟复位

- `clk_i` ：时钟
- `rst_ni` ：低有效复位

2. APB 侧接口

- `apb_psel_i/apb_penable_i/apb_pwrite_i/apb_paddr_i/apb_pwdata_i`
- `apb_prdata_o/apb_pready_o`
- 实现特性：
  - `apb_pready_o` 常为 `1` （零等待）。
  - 不输出 `pslverr` 。
  - 写使能： `wr_en_w = psel & penable & pwrite` 。
  - 读使能： `rd_en_w = psel & ~pwrite` （组合读返回）。

3. 输出到后级控制/数据通路

- 启动与中断： `start_pulse_o` 、 `intr_en_o`
- 模式控制： `mode_o[1:0]` 、 `aes_mode_o[2:0]` 、 `aes_key_len_o[1:0]` 、 `aes_enc_dec_o` 、 `tag_enable_o`
- 地址长度： `src_base_o` 、 `dst_base_o` 、 `msg_len_bytes_o`
- AES 参数： `aes_key_o[0..7]` 、 `aes_iv_o[0..3]`

4. 输入自后级状态

- `busy_i`、`done_i`、`err_i`、`err_code_i[3:0]`、`err_src_i[3:0]`、`state_dbg_i[7:0]`
- `hash_i[0..7]`

3. 寄存器地址映射

说明：下表地址为 `secure_cfg` 内部偏移地址（offset）。 在 SoC 中，软件基址 `SEC_BASE = 0x0000_8000`，实际访问地址为 `SEC_BASE + offset`。

Offset	名称	访问	说明
`0x00`	`CTRL`	RW	启动/中断使能
`0x04`	`MODE_CFG`	RW	模式配置
`0x08`	`SRC_BASE`	RW	源地址
`0x0C`	`DST_BASE`	RW	目的地址
`0x10`	`MSG_LEN`	RW	消息字节长度
`0x14` ~ `0x30`	`KEY0..KEY7`	RW	AES Key（8x32b）
`0x34` ~ `0x40`	`IV0..IV3`	RW	AES IV（4x32b）
`0x44`	`STATUS`	RO	状态与错误
`0x60` ~ `0x7C`	`HASH0..HASH7`	RO	SHA256 摘要（8x32b）

4. 寄存器 bit 定义

4.1 CTRL（`0x00`，RW)

- bit0 `start`（W1P）：写 `1` 触发一次任务启动脉冲 `start_pulse_o`（仅 1 拍）。
- bit2 `intr_en`（RW）：中断使能。
- 其他位：保留。

写 `start=1` 时附带行为：

- 清除 `done_sticky`。
- 清除 `err_sticky`、`err_code_sticky` 与 `err_src_sticky`。

读回值：

- 返回 `{..., intr_en, 0, 0}`，即 `start` 不保持为高电平。

4.2 MODE_CFG（`0x04`，RW)

- bit[1:0] `mode`
  - `0`：AES_ONLY
  - `1`：SHA_ONLY
  - `2`：AES_SHA（组合模式）
  - `3`：保留
- bit[4:2] `aes_mode`
  - `0`：ECB

    - `1`：CBC
    - `2`：CFB（预留）
    - `3`：OFB（预留）
    - `4`：CTR（预留）
    - `5`：GCM（预留）
    - `6`：CCM（预留）
    - `7`：保留
- bit[6:5] `aes_key_len`
    - `0`：AES-128
    - `1`：AES-192（预留）
    - `2`：AES-256
    - `3`：保留
- bit7 `aes_enc_dec`
    - `1`：加密
    - `0`：解密
- bit8 `tag_enable`（AEAD 场景预留）
- bit[31:9]：保留

4.3 SRC_BASE (`0x08`, RW)

- bit[31:0]：DMA 读取源地址（字节地址）

4.4 DST_BASE (`0x0C`, RW)

- bit[31:0]：DMA 写回目的地址（字节地址）

4.5 MSG_LEN (`0x10`, RW)

- bit[31:0]：消息长度（单位：字节）

4.6 KEY0..KEY7 (`0x14` ~ `0x30`, RW)

- 每个寄存器 32 位，共 256 位 key 空间。
- `KEY0` 为最低 offset，`KEY7` 为最高 offset。

4.7 IV0..IV3 (`0x34` ~ `0x40`, RW)

- 每个寄存器 32 位，共 128 位 IV 空间。

4.8 STATUS (`0x44`, RO) 当前代码拼接等价有效位如下（从低到高）：

- bit0：`busy_i`
- bit1：`done_sticky`
- bit2：`err_sticky`
- bit[6:3]：`err_code_sticky`
- bit[10:7]：`err_src_sticky`
- bit[18:11]：`state_dbg_i`
- bit[31:19]：保留（0）

说明：

- `done_sticky` ：收到 `done_i` 后置 1，直到下一次 `start` 清零。
- `err_sticky/err_code_sticky/err_src_sticky` ：收到 `err_i` 后锁存，直到下一次 `start` 清零。

进一步解释：

1. `err_sticky`

- 只是一个“本次任务是否曾经出错”的 sticky 标志。
- 一旦后级在任务执行过程中拉高 `err_i` ，它就保持为 1。
- 这样做的目的，是避免软件因为轮询不及时而错过短脉冲错误事件。

2. `err_code_sticky`

- 锁存出错时的错误类型编码。
- 来源是后级实时输入 `err_code_i[3:0]` 。
- 典型来源包括：
  - `secure_flow_ctrl` 的参数检查错误
  - DMA 读写返回的 AXI 响应错误
  - SHA 路径错误
- 该值不会在 `err_i` 消失后自动清零，而是保持到下一次 `CTRL.start` 。

3. `err_src_sticky`

- 锁存错误来源模块编码。
- 来源是后级实时输入 `err_src_i[3:0]` 。
- 它回答的是“错发生在哪个子路径”，而不是“错的类型是什么”。
- 例如软件看到：
  - `err_code_sticky = 1`
  - `err_src_sticky = 2` 就可以理解成“写回路径报告了错误码 1”。

4. 为什么要同时保留 `err_code_sticky` 和 `err_src_sticky`

- `err_code_sticky` 解决“发生了什么错”
- `err_src_sticky` 解决“是谁报的错”
- 两者一起才能让软件快速定位问题，而不是只有一个笼统的 `err=1`

5. sticky 的清零时机

- 复位时清零
- 软件写 `CTRL.start=1` 时清零
- 任务运行过程中不会自动清零

6. 软件使用建议

- 轮询 `STATUS` 时，若 `bit2 err_sticky = 1`
- 应继续读取：

    - `bit[6:3] err_code_sticky`
    - `bit[10:7] err_src_sticky`
- 然后再决定是：
    - 重新发起任务
    - 还是保留现场继续调试

4.9 HASH0..HASH7 (`0x60` ~ `0x7C` , RO)

- 每个寄存器对应 `hash_i[n]` 。
- 由 SHA 路径计算并回填， `secure_cfg` 仅做寄存器映射。

5. 时序与行为细节

1. `start_pulse_o` 每次写 `CTRL.start=1` 仅拉高 1 个时钟周期。
2. `intr_en_o` 在写 `CTRL` 时更新，保持寄存。
3. `done_i/err_i` 输入采用 sticky 策略，便于软件轮询。
4. 复位后配置寄存器全部清零， `aes_enc_dec_o` 缺省为 `1` （加密）。

6. 软件使用建议

推荐顺序：

1. 写 `MODE_CFG/SRC_BASE/DST_BASE/MSG_LEN/KEY/IV` 。
2. 写 `CTRL` （ `intr_en` 可选， `start=1` ）。
3. 轮询 `STATUS` ：
    - 若 `err=1` ，同时读取 `err_src` 与 `err_code` 做故障定位。
    - 若 `done=1 && busy=0` ，任务完成。
4. 如为 SHA 流程，读取 `HASH0..HASH7` 。

7. 与其他模块边界

- `secure_cfg` 只处理寄存器平面，不参与 DMA 数据搬运。
- `secure_flow_ctrl` 负责流程状态机与完成条件判断。
- `secure_sha_path` 负责哈希计算和 digest 产生。
- `secure_dma_rd/wr` 负责 AXI 数据搬运。

4.2 secure_flow_ctrl 模块设计说明

1. 模块定位

`secure_flow_ctrl` 是 `secure_combo` 的任务流程控制器（control FSM），负责：

1. 接收启动命令 `start_i` 并进行任务参数合法性检查。
2. 根据运行模式（AES_ONLY / SHA_ONLY / AES_SHA）汇总各子路径完成信号。
3. 统一输出运行状态： `busy_o` / `done_o` / `err_o` / `err_code_o` / `err_src_o` / `state_dbg_o` 。

它不搬运数据、不做加解密、不做哈希，仅做“流程编排 + 错误收敛”。

2. 接口定义

```
                                          +----------------------------+
                                          |                            |
clk_i                        ----------->  |                            |
rst_ni                       ----------->  |                            |
start_i                      ----------->  |                            |
mode_i[1:0]                  ----------->  |                            |
aes_mode_i[2:0]              ----------->  |                            |  -----------> bus
aes_key_len_i[1:0]           ----------->  |      secure_flow_ctrl      |  -----------> don
msg_len_bytes_i[31:0]        ----------->  |                            |  -----------> err
aes_done_i                   ----------->  |                            |  -----------> err
sha_done_i                   ----------->  |                            |  -----------> err
wr_done_i                    ----------->  |                            |  -----------> sta
rd_done_i                    ----------->  |                            |
datapath_err_i               ----------->  |                            |
datapath_err_code_i[3:0]     ----------->  |                            |
datapath_err_src_i[3:0]      ----------->  |                            |
                                          |                            |
                                          +----------------------------+
```

2.1 输入

- `clk_i` / `rst_ni` ：时钟与低有效复位
- `start_i` ：启动脉冲（来自 `secure_cfg` ）
- `mode_i[1:0]` ：任务模式
  - `2'd0` ：AES_ONLY
  - `2'd1` ：SHA_ONLY
  - `2'd2` ：AES_SHA
- `aes_mode_i[2:0]` ：AES 工作模式编码（来自 `MODE_CFG[4:2]` ）
- `aes_key_len_i[1:0]` ：AES 密钥长度编码（来自 `MODE_CFG[6:5]` ）
- `msg_len_bytes_i[31:0]` ：消息长度（字节）
- 子路径完成信号：
  - `rd_done_i` ：读 DMA 完成
  - `aes_done_i` ：AES 处理完成
  - `wr_done_i` ：写 DMA 完成
  - `sha_done_i` ：SHA 处理完成
- `datapath_err_i` ：数据通路错误汇总输入
- `datapath_err_code_i[3:0]` ：数据通路细分错误码（由 top 聚合）
- `datapath_err_src_i[3:0]` ：数据通路错误来源（由 top 聚合）

2.2 输出

- `busy_o` ：任务运行中
- `done_o` ：任务完成脉冲（1 拍）
- `err_o` ：错误状态（sticky，直到下次 start 清除）

- `err_code_o[3:0]` ：错误码
- `err_src_o[3:0]` ：错误来源码
- `state_dbg_o[7:0]` ：当前状态（调试）

3. 状态机设计

状态定义：

- `ST_IDLE (0)` ：空闲，等待 `start_i`
- `ST_RUN (1)` ：运行中，等待完成或错误
- `ST_DONE (2)` ：完成态，仅 1 拍后回 `IDLE`
- `ST_ERR (3)` ：错误态，等待下一次 `start_i` 清错并回 `IDLE`

`state_dbg_o` 直接输出上述编码。

4. 启动与合法性检查

在 `ST_IDLE` 接收到 `start_i` 后执行：

1. 长度为 0 检查

- 条件： `msg_len_bytes_i == 0`
- 结果：进入 `ST_ERR`
- 错误码： `ERR_ZERO_LEN = 4'd3`

2. AES 对齐检查（仅模式包含 AES 时）

- 条件： `mode_has_aes_w && (msg_len_bytes_i[3:0] != 4'h0)`
- 结果：进入 `ST_ERR`
- 错误码： `ERR_AES_ALIGN = 4'd1`

3. AES 模式支持性检查（仅模式包含 AES 时）

- 条件： `mode_has_aes_w && !aes_mode_supported_w` （当前仅支持 ECB/CBC）
- 结果：进入 `ST_ERR`
- 错误码： `ERR_AES_MODE_UNSUP = 4'd4`

4. AES 密钥长度支持性检查（仅模式包含 AES 时）

- 条件： `mode_has_aes_w && !aes_key_supported_w` （当前仅支持 128/256）
- 结果：进入 `ST_ERR`
- 错误码： `ERR_AES_KEY_UNSUP = 4'd5`

5. 检查通过

- 进入 `ST_RUN`
- `busy_o = 1`
- 清空本次运行的 `*_done_seen_q`

5. 完成条件收敛

模块使用 done 锁存位（`*_done_seen_q`）对脉冲完成信号去抖与记忆：

- `rd_done_seen_q`
- `aes_done_seen_q`
- `wr_done_seen_q`
- `sha_done_seen_q`

在 `busy_o==1` 期间，一旦对应 `*_done_i` 拉高，该 seen 位保持为 1，直到下一次任务启动时清零。

这样可以避免“done 脉冲过窄导致组合判断丢失”的问题。

5.1 各模式完成判据

- AES_ONLY（`mode_i==0`）：
  - `rd_done_seen_q && aes_done_seen_q && wr_done_seen_q`
- SHA_ONLY（`mode_i==1`）：
  - `rd_done_seen_q && sha_done_seen_q`
- AES_SHA（`mode_i==2`）：
  - `rd_done_seen_q && aes_done_seen_q && wr_done_seen_q && sha_done_seen_q`

满足后：

- 进入 `ST_DONE`
- `busy_o = 0`
- `done_o = 1`（单拍）

6. 错误处理

6.1 错误优先级 在 `ST_RUN` 中，`datapath_err_i` 优先于完成判定：

- 若 `datapath_err_i==1`：
  - 立即进入 `ST_ERR`
  - `busy_o=0`
  - `err_o=1`
  - `err_code_o=datapath_err_code_i`
  - `err_src_o=datapath_err_src_i`

6.2 错误码定义

- `4'd0`：`ERR_NONE`（无错误）
- `4'd1`：`ERR_AES_ALIGN`（AES 模式下长度非 16 字节对齐）
- `4'd3`：`ERR_ZERO_LEN`（消息长度为 0）
- `4'd4`：`ERR_AES_MODE_UNSUP`（AES 模式编码当前不支持）
- `4'd5`：`ERR_AES_KEY_UNSUP`（AES 密钥长度编码当前不支持）

数据通路错误码（来自 top 聚合）不再被折叠为统一 `ERR_DATAPATH`，而是直接透传到 `err_code_o`。

6.3 错误来源码定义

- `4'd0` ：`ERR_SRC_NONE`（无错误）
- `4'd1` ：`ERR_SRC_FLOW`（flow_ctrl 参数检查错误）
- 其他值由 top 传入（例如 RD DMA / WR DMA / SHA 路）

6.4 错误态退出策略

- `ST_ERR` 不自动退出。
- 只有再次收到 `start_i` 才清错并回 `ST_IDLE`。

设计原因：

- 让软件有充分时间读取 `err_o/err_code_o`。
- 避免错误在一个时钟后被覆盖，导致调试困难。

7. 输出时序特征

1. `busy_o`

- 任务进入 `ST_RUN` 时置 1。
- 完成或错误退出运行态时清 0。

2. `done_o`

- 只在完成当拍脉冲拉高 1 个时钟。
- 下一拍进入 `ST_DONE -> ST_IDLE` ，`done_o` 自动回 0。

3. `err_o`

- 进入 `ST_ERR` 后保持为 1。
- 下一次 `start_i` 时清 0。

4. `err_code_o` / `err_src_o`

- 报错时锁定为对应错误码。
- 下一次 `start_i` 时恢复 `ERR_NONE`。

8. 与 secure_cfg 的配合关系

- `secure_cfg` 产生 `start_pulse_o` 给 `secure_flow_ctrl.start_i`。
- `secure_cfg` 读取 `busy_o/done_o/err_o/err_code_o/err_src_o/state_dbg_o` 映射到 `STATUS` 寄存器。
- `secure_cfg` 本身不做状态判定，只负责寄存器化。

9. 软件可见语义（轮询建议）

推荐轮询逻辑：

1. 发起 start。
2. 读取 `STATUS` ：
    - `err==1` ：立即失败，同时读取 `err_src + err_code` 定位原因。
    - `done==1 && busy==0` ：任务成功完成。
3. 超时防护：软件应设定轮询超时，避免死等。

4.3 secure_dma_rd 模块设计说明

1. 模块定位

`secure_dma_rd` 是 `secure_combo` 的 AXI 读 DMA 引擎，职责是：

1. 接收一条“读描述符”（源地址 + 长度 + tag）。
2. 生成一组 AXI 读突发请求（AR），从内存读取数据。
3. 将 AXI R 通道数据转换为 AXIS 流（ `tdata/tvalid/tlast` ）输出给后级。
4. 在本描述符最后一个输出 beat 时回报状态（tag/error/valid）。

它只负责“内存 -> AXIS”的数据搬运，不做 AES/SHA 算法计算。

2. 接口定义

```
                                                  +----------------------------+
                                                  |                            |
clk_i                            ----------->     |                            |
rst_ni                           ----------->     |                            |
s_axis_read_desc_addr_i[31:0]    ----------->     |                            |  ----------->
s_axis_read_desc_len_i[19:0]     ----------->     |                            |  ----------->
s_axis_read_desc_tag_i[7:0]      ----------->     |                            |  ----------->
s_axis_read_desc_valid_i         ----------->     |                            |  ----------->
m_axis_read_data_tready_i        ----------->     |                            |  ----------->
m_axi_arready_i                  ----------->     |                            |  ----------->
m_axi_rdata_i[31:0]              ----------->     |        secure_dma_rd       |  ----------->
m_axi_rresp_i[1:0]               ----------->     |                            |  ----------->
m_axi_rlast_i                    ----------->     |                            |  ----------->
m_axi_rvalid_i                   ----------->     |                            |  ----------->
enable_i                         ----------->     |                            |  ----------->
                                                  |                            |  ----------->
                                                  |                            |  ----------->
                                                  +----------------------------+
```

2.1 时钟复位与使能

- `clk_i` / `rst_ni` ：时钟与低有效复位
- `enable_i` ：模块使能（关闭时不接收新描述符）

2.2 描述符输入（AXIS-like command）

- `s_axis_read_desc_addr_i[AXI_ADDR_WIDTH-1:0]` ：源地址（字节地址）
- `s_axis_read_desc_len_i[LEN_WIDTH-1:0]` ：长度编码，语义为 `bytes-1`

- `s_axis_read_desc_tag_i[TAG_WIDTH-1:0]` ：用户 tag
- `s_axis_read_desc_valid_i` / `s_axis_read_desc_ready_o` ：握手

2.3 描述符状态输出

- `m_axis_read_desc_status_tag_o` ：回传原 tag
- `m_axis_read_desc_status_error_o[3:0]` ：错误码（按 `RRESP` 映射）
- `m_axis_read_desc_status_valid_o` ：状态有效脉冲

2.4 AXIS 数据输出

- `m_axis_read_data_tdata_o[AXIS_DATA_WIDTH-1:0]`
- `m_axis_read_data_tvalid_o`
- `m_axis_read_data_tready_i`
- `m_axis_read_data_tlast_o`

2.5 AXI 读主接口

- AR 通道：
  `m_axi_araddr_o/m_axi_arlen_o/m_axi_arsize_o/m_axi_arburst_o/m_axi_arvalid_o/m_axi_arready_i`
- R 通道： `m_axi_rdata_i/m_axi_rresp_i/m_axi_rlast_i/m_axi_rvalid_i/m_axi_rready_o`

3. 参数与基本约定

- `AXI_DATA_WIDTH=32` （默认）
- `AXI_BURST_SIZE = clog2(AXI_DATA_WIDTH/8)` ，32-bit 时为 2（每拍 4 字节）
- `AXI_MAX_BURST_LEN=16` （最大 16 beats）
- `AXI_MAX_BURST_SIZE = AXI_MAX_BURST_LEN << AXI_BURST_SIZE` （默认 64 字节）

长度约定：

- 输入描述符 `len` 使用 `bytes-1` 编码。
- 模块内部通过 `len + 1` 还原真实字节数。

4. 高层数据通路

描述符进入后，模块分两条并行控制链：

1. AXI 请求链（AXI FSM）

- 根据剩余字节数与地址，持续切分出多个 AR burst。
- 满足 AXI 边界限制（含 4KB 边界约束）。

2. AXIS 输出链（AXIS FSM）

- 接收 AXI R 数据。
- 经过内部输出 FIFO 后，对外以 AXIS 协议输出。
- 在最后一个输出 beat 打 `tlast` ，并回报描述符状态。

两条链通过 `axis_cmd_*` 寄存器解耦。

5. 描述符处理流程

5.1 接收描述符 条件：

- `enable_i==1`
- 当前无待执行 axis 命令（ `axis_cmd_valid_reg==0` ）

握手成功后：

- 锁存 `addr/tag`。
- 计算总字节数 `op_bytes_count = len + 1`。
- 计算预计输入/输出 beat 计数。
- 置 `axis_cmd_valid=1` ，供 AXIS FSM 领取。

5.2 地址与长度递推 每次准备发一个 burst 时：

- 计算本 burst 的传输字节数 `tr_bytes_count`。
- 更新：
  - `addr += tr_bytes_count`
  - `op_bytes_count -= tr_bytes_count`
- 若仍有剩余字节，继续下一个 burst。

6. AXI Burst 切分规则

模块在每个 burst 计算时遵守两类约束：

1. 最大突发长度限制

- 不超过 `AXI_MAX_BURST_SIZE`（默认 64 字节）

2. 4KB 边界限制

- 一个 burst 不能跨越 4KB 边界。
- 若本次请求会跨界，则截断到 `4KB - (addr[11:0])`。

最终：

- `m_axi_araddr_o = 当前地址`
- `m_axi_arlen_o = (tr_bytes_count-1) >> AXI_BURST_SIZE`（beats-1）
- `m_axi_arsize_o = AXI_BURST_SIZE`
- `m_axi_arburst_o = INCR(2'b01)`

7. AXIS 输出与 backpressure

7.1 内部输出 FIFO 模块带一个输出 FIFO（ `OUTPUT_FIFO_ADDR_WIDTH=5` ，深度 32 beat）：

- 作用：解耦 AXI R 数据到达节奏与下游 AXIS 消费节奏。

- `m_axis_read_data_tready_int = !out_fifo_half_full_reg`
- FIFO 半满后会反压 AXI R（降低/拉低 `m_axi_rready_o`）

7.2 不丢数机制

- 仅在 FIFO 可写时接收/缓存 `m_axis_read_data_tvalid_int`。
- 下游 `m_axis_read_data_tready_i` 低时，`m_axis_read_data_tvalid_reg` 保持，数据驻留不丢失。
- 因为 AXI R 握手受 `m_axi_rready_o` 控制，源头可被反压。

7.3 `tlast` 生成

- 使用内部输出计数 `output_cycle_count` 判定最后一个输出 beat。
- 最后一拍设置 `m_axis_read_data_tlast_o=1`。

8. 状态机说明

模块包含两个 FSM。

8.1 AXI FSM 状态：

- `AXI_STATE_IDLE`：等待新描述符
- `AXI_STATE_START`：计算并发 AR，循环直到本描述符全部字节都被分配完

关键点：

- `m_axi_arvalid_o` 采用“保持到握手”策略（valid-sticky until ready）。

8.2 AXIS FSM 状态：

- `AXIS_STATE_IDLE`：等待 `axis_cmd_valid` 被领取
- `AXIS_STATE_READ`：消费 AXI R 数据并向输出 FIFO 推送

关键点：

- 仅当 `m_axi_rready_o && m_axi_rvalid_i` 才算接收一个输入 beat。
- 最后一拍同时触发：
  - `tlast`
  - `m_axis_read_desc_status_valid_o=1`

9. 描述符状态回报

在该描述符最后一个输出 beat 时：

- `m_axis_read_desc_status_tag_o = 原 tag`
- `m_axis_read_desc_status_error_o = 本描述符累计错误码`
- `m_axis_read_desc_status_valid_o = 1`（单拍）

错误码映射（`RRESP -> status_error`）：

1. `2'b00 (OKAY)` -> `4'd0`
2. `2'b10 (SLVERR)` -> `4'd1`
3. `2'b11 (DECERR)` -> `4'd2`
4. 其他值（例如 `EXOKAY`）-> `4'd3`

说明：

1. 错误码在 descriptor 开始时清零。
2. 同一 descriptor 内若出现多个错误响应，模块保留首个非零错误码。

10. 与 secure_combo 的协同

`secure_combo_top` 中：

- 描述符来源：`src_base` + `msg_len-1`
- AXIS 输出送入 `secure_axis_split`
- `rd_stat_valid/rd_stat_err` 反馈给 `secure_flow_ctrl`

在 `flow_ctrl` 视角：

- `rd_done_i` 本质来自 `m_axis_read_desc_status_valid_o`

11. 软件/系统使用注意事项

1. `msg_len` 必须由上层保证合法（非 0，且与模式匹配）。
2. 地址建议按总线宽度对齐（32-bit 系统通常 4 字节对齐）。
3. 大块传输会被自动拆分为多次 burst（受 max burst 与 4KB 边界约束）。
4. 下游阻塞时，DMA 会自动反压，不应丢数据，但会拉长总时延。

当前实现里几个容易在综合报告里看到的点

1. `m_axi_rlast_i`

- 当前 RTL 里没有参与功能判断。
- 读完成判定依赖内部 beat 计数，而不是依赖 AXI `RLAST`。
- 所以 `check_design` 会把它标成 unconnected port。

2. `bytes-1` 长度编码

- `s_axis_read_desc_len_i` 输入是 `bytes-1`
- 模块进入内部后第一步会恢复成真实字节数 `len + 1`

3. 输出 FIFO

- 这是为了吸收 `AXI R` 与下游 `AXIS ready` 的节拍差
- 也使得 `m_axi_rready_o` 能按 FIFO 空间进行反压控制

4.4 secure_axis_split 模块设计说明

1. 模块定位

`secure_axis_split` 是 `secure_combo` 数据路径中的 AXIS 分流器（broadcast splitter）。

它的职责是：

1. 接收一条上游 AXIS 数据流。
2. 按使能位将同一拍数据复制到 0 号路径（AES）和/或 1 号路径（SHA）。
3. 通过统一 ready 规则保证“被使能路径的数据一致性与拍对齐”。

该模块不缓存数据、不重排数据、不改写 payload，只做“条件广播 + 握手门控”。

2. 接口定义

```
                                          +----------------------------+
                                          |                            |
path0_en_i                    ----------> |                            | ----------> s_a
path1_en_i                    ----------> |                            | ----------> m0_
s_axis_data_i[31:0]           ----------> |                            | ----------> m0_
s_axis_valid_i                ----------> |     secure_axis_split      | ----------> m0_
s_axis_last_i                 ----------> |                            | ----------> m1_
m0_axis_ready_i               ----------> |                            | ----------> m1_
m1_axis_ready_i               ----------> |                            | ----------> m1_
                                          |                            |
                                          +----------------------------+
```

2.1 使能输入

- `path0_en_i`：0 号输出路径使能（通常接 AES 路）
- `path1_en_i`：1 号输出路径使能（通常接 SHA 路）

2.2 上游 AXIS 从口（输入）

- `s_axis_data_i[DATA_W-1:0]`
- `s_axis_valid_i`
- `s_axis_ready_o`
- `s_axis_last_i`

2.3 下游 AXIS 主口（输出）

- 0 号路径：
  - `m0_axis_data_o`
  - `m0_axis_valid_o`
  - `m0_axis_ready_i`
  - `m0_axis_last_o`
- 1 号路径：
  - `m1_axis_data_o`
  - `m1_axis_valid_o`

- `m1_axis_ready_i`
- `m1_axis_last_o`

3. 核心行为

3.1 数据复制（data/last）

- `m0_axis_data_o = s_axis_data_i`
- `m1_axis_data_o = s_axis_data_i`
- `m0_axis_last_o = s_axis_last_i`
- `m1_axis_last_o = s_axis_last_i`

即：被广播的两路看到完全相同的 payload 和 `last`。

3.2 有效位门控（valid）

- `m0_axis_valid_o = s_axis_valid_i & path0_en_i`
- `m1_axis_valid_o = s_axis_valid_i & path1_en_i`

未使能路径 `valid=0`，相当于该路径本拍无事务。

3.3 就绪位聚合（ready）模块最关键的一条：

```
s_axis_ready_o = (path0_en_i ? m0_axis_ready_i : 1'b1) & (path1_en_i ? m1_axis_ready_i : 1'b1)
```

含义：

- 只要某条“使能路径”没 ready，上游就不能前进。
- 未使能路径不参与阻塞（按 ready=1 处理）。

4. 时序特征

该模块是纯组合逻辑（无寄存器）：

- 无额外时延（逻辑路径短）。
- 无内部缓存，反压实时透传到上游。

因此它与前后级 FIFO 配合使用，避免长组合链造成时序压力。

5. 在 secure_combo 中的作用

在 `secure_combo_top` 中，`secure_axis_split` 位于：

```
DMA_RD 输出 -> AXIS_SPLIT -> AES_FIFO 路 + SHA_FIFO 路
```

用途：

- AES_ONLY：只开 AES 路
- SHA_ONLY：只开 SHA 路

- AES_SHA：双路同时开，保证同一份输入数据同时喂给 AES 与 SHA

6. 设计边界与注意事项

1. 模块不缓存：下游 backpressure 会直接阻塞上游。
2. 模块不改包：`last` 直接复制，不做包长重构。
3. 若双路使能，吞吐取决于“较慢那一路”。
4. 若不希望慢路拖慢快路，需要在系统层加入异步队列/复制缓存策略（本模块不负责）。

4.5 secure_fifo 模块设计说明

1. 模块定位

`secure_fifo` 是 `secure_combo` 数据路径中的通用同步 FIFO（单时钟域），用于在上下游之间提供弹性缓冲。

主要作用：

1. 解耦上下游处理速率。
2. 在下游阻塞时对上游施加反压，防止数据丢失。
3. 支持持续吞吐场景中的同拍入队/出队。

该模块不修改数据内容，不做包处理逻辑，只做“按拍缓存和流控”。

2. 接口定义

```
                                          +----------------------------+
                                          |                            |
clk_i                        ----------->  |                            |  -----------> s_r
rst_ni                       ----------->  |                            |  -----------> m_d
clr_i                        ----------->  |        secure_fifo         |  -----------> m_v
s_data_i[32:0]               ----------->  |                            |
s_valid_i                    ----------->  |                            |
m_ready_i                    ----------->  |                            |
                                          |                            |
                                          +----------------------------+
```

2.1 时钟复位

- `clk_i` ：时钟
- `rst_ni` ：低有效复位

2.2 上游输入侧（source side）

- `s_data_i[DATA_W-1:0]` ：输入数据
- `s_valid_i` ：输入有效
- `s_ready_o` ：FIFO 可接收（反压信号）

2.3 下游输出侧（sink side）

- `m_data_o[DATA_W-1:0]` ：输出数据
- `m_valid_o` ：输出有效
- `m_ready_i` ：下游可接收

默认参数：

- `DATA_W = 33`
- `AW = 4` ，深度 `DEPTH = 2^AW = 16`

3. 内部结构

3.1 存储体

- `mem_q[0:DEPTH-1]` ：FIFO RAM

3.2 指针

- `wr_ptr_q[AW:0]` ：写指针（含 1 个翻转位）
- `rd_ptr_q[AW:0]` ：读指针（含 1 个翻转位）

采用“扩展 1 位”的环形指针方案：

- 低 `AW` 位用于 RAM 地址。
- 高 1 位用于区分“同地址但不同圈数”，从而区分 full 与 empty。

4. 满空判定

4.1 空（empty） `fifo_empty_w = (wr_ptr_q == rd_ptr_q)`

含义：写读指针完全相等，没有可读数据。

4.2 满（full） `fifo_full_w = (wr_ptr_q[AW] != rd_ptr_q[AW]) && (wr_ptr_q[AW-1:0] == rd_ptr_q[AW-1:0])`

含义：地址位相同但翻转位不同，表示写指针“追上”读指针一整圈，FIFO 满。

5. AXIS-like 握手语义

5.1 可接收/可输出

- `s_ready_o = ~fifo_full_w`
- `m_valid_o = ~fifo_empty_w`

为什么 `m_valid_o` 只由 empty 决定

- `m_valid_o` 表示“FIFO 当前是否有数据可供读取”，这是源端自身状态，应独立于下游 `m_ready_i` 。
- 若 `valid` 反向依赖 `ready` ，会导致组合耦合与潜在死锁风险。

5.2 入队/出队触发

- `push_fire_w = s_valid_i & s_ready_o`
- `pop_fire_w = m_valid_o & m_ready_i`

只有握手成功才推进对应指针。

6. 时序行为

在 `posedge clk_i` ：

1. 若 `push_fire_w` ：

- `mem_q[wr_ptr_q[AW-1:0]] <= s_data_i`
- `wr_ptr_q <= wr_ptr_q + 1`

2. 若 `pop_fire_w` ：

- `rd_ptr_q <= rd_ptr_q + 1`

3. 复位：

- `wr_ptr_q = 0`
- `rd_ptr_q = 0`

7. 同拍 push 与 pop

该 FIFO 允许同一拍同时发生 push 与 pop（只要握手条件都满足）。

收益：

1. 提高吞吐：在稳态流式场景可实现近似每拍一进一出。
2. 降低阻塞：避免“先出再进/先进再出”的人工气泡。
3. 语义清晰：深度净变化为 0，但数据流持续推进。

8. 在 secure_combo 中的典型用途

`secure_combo_top` 中有两处实例：

- `u_fifo_aes` ：分流后 AES 路缓冲
- `u_fifo_sha` ：分流后 SHA 路缓冲

目的：

- 吸收 `secure_axis_split` 输出与 AES/SHA 消费速率差。
- 通过 backpressure 把拥塞向上游安全传播，避免丢拍。

9. 设计边界

1. 单时钟 FIFO：不支持异步跨时钟域。
2. 无“almost_full/almost_empty”阈值输出。
3. 无 ECC/奇偶校验。
4. 无包级语义（ `tlast` 只是数据位的一部分，若需要可随数据一并存入）。

4.6 secure_sha_path 设计说明（流式分块版）

1. 模块定位

`secure_sha_path` 是 `secure_combo` 内部的 SHA 数据路径控制器，负责把上游输入的原始消息整理成 `sha256_core` 可直接消费的 512-bit block 流。

1. 接收上游通过 AXIS 输入的原始消息数据。
2. 在模块内部完成 SHA-256 标准 padding。
3. 按 block 为单位把 16 个 32-bit word 送入 `sha256_core`。
4. 接收 digest，并向上层给出 `done_o/err_o`。

2. 接口说明

```
                                          +----------------------------+
                                          |                            |
clk_i                        ----------->  |                            |
rst_ni                       ----------->  |                            |
start_i                      ----------->  |                            |  -----------> s_a
msg_len_bytes_i[31:0]        ----------->  |      secure_sha_path       |  -----------> dig
s_axis_data_i[31:0]          ----------->  |                            |  -----------> dig
s_axis_valid_i               ----------->  |                            |  -----------> don
s_axis_last_i                ----------->  |                            |  -----------> err
                                          |                            |
                                          +----------------------------+
```

2.1 输入接口

1. `clk_i`，`rst_ni`：时钟与低有效复位。
2. `start_i`：启动一次新的 SHA 消息处理。
3. `msg_len_bytes_i[31:0]`：本次消息长度，单位是 byte。模块按这个长度收包，而不是依赖 `s_axis_last_i` 来判定结束。
4. `s_axis_data_i[31:0]`：上游输入数据，每拍 32bit，即 4 个字节。在本模块中按 byte 写入 block buffer。
5. `s_axis_valid_i`：输入有效。
6. `s_axis_last_i`：上游最后一拍标记。当前实现**不依赖它来结束收包**，结束条件完全由 `msg_len_bytes_i` 决定。

2.2 输出接口

1. `s_axis_ready_o` ：本模块可接收原文输入时拉高。仅在 `ST_RECV` 状态为 1。
2. `digest_o[255:0]` ： `sha256_core` 最终输出的 digest。
3. `digest_valid_o` ：digest 有效脉冲。
4. `done_o` ：本次消息处理完成脉冲。
5. `err_o` ：错误标志。目前只在启动参数非法时置位。

2.3 与 sha256_core 的内部连接

命令接口

1. `cmd_valid_w` ：在 `ST_CMD` 状态拉高，向 `sha256_core` 发送“本 block 开始处理”的命令。
2. `cmd_init_w` ：当 `blk_idx_q == 0` 时为 1，表示当前是整条消息的第一个 block。
3. `cmd_last_w` ：当 `send_last_block_q == 1` 时为 1，表示当前是最后一个 block。
4. `cmd_ready_w` ：。。 `sha256_core` 返回 ready，命令握手完成后进入发送 word 阶段。

数据接口

1. `core_axis_data_w = core_word_q` ：当前送给 `sha256_core` 的 32-bit word。
2. `core_axis_valid_w` ：在 `ST_HASH_SEND` 状态拉高。
3. `core_axis_last_w` ：当 `word_idx_q == 15` 时为 1，表示当前是本 block 的最后一个 word。
4. `core_axis_ready_w` ： `sha256_core` 返回 ready。

结果接口

1. `sha_digest_w`
2. `sha_digest_valid_w`

3. 内部结构

3.1 核心寄存器

1. `state_q[2:0]` ：主状态机。
2. `msg_len_q[31:0]` ：锁存本次消息长度。
3. `bit_len_q[63:0]` ：消息 bit 长度，即 `msg_len_bytes_i << 3` 。后续会写到最后 8 字节长度域中。
4. `bytes_recv_q[31:0]` ：当前已接收的原文字节数。
5. `blk_idx_q[31:0]` ：当前处理到第几个 SHA block。

6. `block_byte_count_q[6:0]` ：当前 block buffer 中已有多少个有效字节。范围 `0 ~ 64` 。

7. `word_idx_q[4:0]` ：当前 block 内送到了第几个 32-bit word。范围 `0 ~ 15` 。

8. `follow_kind_q[1:0]` ：用来表示“当前 block 发完之后，是否还要自动生成一个跟随块”。编码如下：

    - `FOLLOW_NONE`
    - `FOLLOW_PAD_WITH_80`
    - `FOLLOW_LEN_ONLY`

9. `send_last_block_q` ：当前待发送 block 是否是整条消息的最后一个 block。

10. `core_word_q[31:0]` ：当前准备送入 `sha256_core` 的 word。用于把“当前拍发出去的数据”锁住，避免首拍/切拍不稳定。

11. `block_buf_q[0:63]` ：当前 64-byte block buffer。

3.2 辅助函数

1. `min4_bytes(bytes_left)` ：计算当前这拍最多还应接收多少个有效字节。返回 `1~4` ，或者最后一拍的小于 4 字节值。

2. `block_word(word_idx)` ：从 `block_buf_q` 里取出第 `word_idx` 个 32-bit word。返回格式为 `{byte0, byte1, byte2, byte3}` 即按 SHA 需要的大端 word 顺序拼接。

4. 状态机详细说明

状态定义：

1. `ST_IDLE` ：空闲态
2. `ST_RECV` ：接收原文，并写入当前 block buffer
3. `ST_CMD` ：向 `sha256_core` 发送 block 命令
4. `ST_WAIT_RDY` ：等待 `sha256_core` 的数据通道 ready，并预装首词
5. `ST_HASH_SEND` ：把当前 block 的 16 个 word 顺序送入 `sha256_core`
6. `ST_WAIT_LAST` ：等待 `sha256_core` 返回 digest
7. `ST_DONE` ：完成态
8. `ST_ERR` ：错误态

4.1 ST_IDLE

1. 等待 `start_i` 。
2. 收到启动后锁存：
    - `msg_len_q`
    - `bit_len_q`
    - 清零 `bytes_recv_q`
    - 清零 `blk_idx_q`
    - 清零 `block_byte_count_q`

- 清零 `word_idx_q`
- 清零 `follow_kind_q`
- 清零 `send_last_block_q`
- 清零 `core_word_q`
- 清空 `block_buf_q`

3. 参数检查：
   - `msg_len == 0` 或 `msg_len > MAX_MSG_BYTES` -> `ST_ERR`
   - 其他情况 -> `ST_RECV`

4.2 ST_RECV

握手条件：`input_fire_w = s_axis_valid_i & s_axis_ready_o`

内部动作：

1. 先用 `min4_bytes()` 算出当前拍实际有效字节数。
2. 将 `s_axis_data_i[7:0] / [15:8] / [23:16] / [31:24]` 依次写入 `block_buf_q`。
3. 更新：
   - `bytes_recv_q`
   - `block_byte_count_q`

随后按情况进行分支：

1. 消息在当前拍结束，且刚好落在 64-byte 边界

   - `bytes_recv_q + valid_bytes == msg_len_q`
   - `next_block_count_r == 64`

   说明：

   - 当前 block 纯原文刚好填满。
   - 但 SHA 还需要额外一个 padding block。

   处理：

   - `follow_kind_q <= FOLLOW_PAD_WITH_80`
   - `send_last_block_q <= 0`
   - 转 `ST_CMD`

2. 消息在当前拍结束，且当前 block 还能容纳 padding 和长度域

   - `bytes_recv_q + valid_bytes == msg_len_q`
   - `next_block_count_r <= 55`

   处理：

   1. 在 `block_buf_q[next_block_count_r]` 写入 `8'h80`
   2. 中间未使用部分补 `0`
   3. `block_buf_q[56:63]` 写入 `bit_len_q[63:0]`

    4. `send_last_block_q <= 1`
    5. 转 `ST_CMD`

3. 消息在当前拍结束，但当前 block 放不下长度域

    - `bytes_recv_q + valid_bytes == msg_len_q`
    - `56 <= next_block_count_r < 64`

    处理：

    1. 先在本块写入 `8'h80`
    2. 剩余字节补 `0`
    3. 本块先发出去，但它不是最后一个 block
    4. `follow_kind_q <= FOLLOW_LEN_ONLY`
    5. 转 `ST_CMD`

4. 还没到消息结尾，但当前 block 恰好装满 64B

    - `next_block_count_r == 64`
    - 但消息尚未结束

    处理：

    1. 这是一个普通中间块。
    2. `send_last_block_q <= 0`
    3. `follow_kind_q <= FOLLOW_NONE`
    4. 转 `ST_CMD`

4.3 ST_CMD

内部动作：

1. `cmd_valid_w = 1`
2. `cmd_init_w = (blk_idx_q == 0)`
3. `cmd_last_w = send_last_block_q`
4. 等待 `cmd_ready_w`
5. 命令握手完成后：
    - `word_idx_q <= 0`
    - 转 `ST_WAIT_RDY`

4.4 ST_WAIT_RDY

内部动作：

1. 等待 `core_axis_ready_w`
2. ready 到来时：
    - `core_word_q <= block_word(0)`
    - 转 `ST_HASH_SEND`

这个状态存在的工程意义非常明确：

1. 避免首拍数据与 ready 同拍组合生成。
2. 让第一个送出的 word 提前锁存在 `core_word_q` 中。
3. 降低首拍竞争/冒险风险。

4.5 ST_HASH_SEND

握手条件：`core_fire_w = core_axis_valid_w & core_axis_ready_w`

内部动作：

1. 普通 word（`word_idx_q < 15`）

    1. 当前拍发出 `core_word_q`
    2. `word_idx_q <= word_idx_q + 1`
    3. `core_word_q <= block_word(word_idx_q + 1)`

    这就是“当前拍发当前词，同时预装下一词”的结构。

2. 最后一个 word（`word_idx_q == 15`）

    1. 当前 block 就是最后一个 block：`send_last_block_q == 1`。

        处理：转 `ST_WAIT_LAST`

    2. 当前 block 之后还需要跟随块：`follow_kind_q != FOLLOW_NONE`

        处理：

        1. 清空 `block_buf_q`
        2. 若 `FOLLOW_PAD_WITH_80`：在 `block_buf_q[0]` 写 `8'h80`
        3. 在 `block_buf_q[56:63]` 写入 `bit_len_q`
        4. `send_last_block_q <= 1`
        5. `follow_kind_q <= FOLLOW_NONE`
        6. 转 `ST_CMD`

    3. 当前 block 是普通中间块：`follow_kind_q == FOLLOW_NONE` 且不是最后一块 处理：

        1. 清空 `block_buf_q`
        2. `block_byte_count_q <= 0`
        3. 转回 `ST_RECV`

4.6 ST_WAIT_LAST

内部动作：

1. 等待 `sha_digest_valid_w`
2. 有效时：

- `digest_o <= sha_digest_w`
- `digest_valid_o <= 1`
- 转 `ST_DONE`

4.7 ST_DONE

内部动作：

1. `done_o <= 1`
2. 下一拍回 `ST_IDLE`

4.8 ST_ERR

当前错误来源只有两类：

1. `msg_len_bytes_i == 0`
2. `msg_len_bytes_i > MAX_MSG_BYTES`

内部动作：

1. `err_o = 1`
2. 等待新的 `start_i`
3. 收到后清错并回 `ST_IDLE`

5. Padding 处理的关键场景

5.1 原文长度正好是 64B 的整数倍

例如消息长度 = 64B：

1. 第一个 block 全是原文，没有地方插入 `0x80`
2. 所以必须额外生成一个新 block：

- byte0 = `0x80`
- byte1~55 = `0x00`
- byte56~63 = `bit_len`

对应实现就是：`FOLLOW_PAD_WITH_80`

5.2 原文最后一个 block 剩余空间足够容纳长度域

例如最后块只用了 `0~55` 字节：

1. 在末尾插入 `0x80`
2. 中间补 `0`
3. 末尾 `8` 字节写 `bit_len`
4. 只需一个最后块

5.3 原文最后一个 block 剩余空间不足以放长度域

例如最后块已经用了 `56~63` 字节：

1. 本块只能放 `0x80` 和若干 `0`
2. 长度域必须放到下一块的最后 8 字节
3. 因此要自动生成一个“只放长度域的跟随块”

对应实现就是： `FOLLOW_LEN_ONLY`

5.4 不会出现“还剩一点空当，但放不下 `8'h80` ”的情况

当前工程里：

1. 消息长度单位是 `byte`
2. `block_buf_q` 也是按 `byte` 组织
3. padding 起始字节固定写入 `8'h80`

因此这里不存在“当前 block 还有一点空间，但不够放一个 `8'h80` ”的问题。

原因是：

1. `8'h80` 本身就是 1 个完整字节
2. 当前 block 的剩余空间统计单位也是字节
3. 所以只要还有 `1 byte` 空位，就一定能放下 `8'h80`

4.7 secure_dma_wr 模块设计说明

1. 模块定位

`secure_dma_wr` 是 secure_combo 的 AXI 写 DMA 引擎，负责把 AXIS 输入数据写入内存。

核心功能：

1. 接收写描述符（目的地址 + 长度）。
2. 从 AXIS 接收写数据流并缓存到内部 FIFO。
3. 生成 AXI `AW/W/B` 事务完成内存写。
4. 支持 4KB 边界切分和最大 burst 长度限制。
5. 输出写完成状态与错误码。

2. 接口说明

```
                                                  +----------------------------+
                                                  |                            |
clk_i                            ----------->     |                            |
rst_ni                           ----------->     |                            |
s_axis_write_desc_addr_i[31:0]   ----------->     |                            |  ----------->
s_axis_write_desc_len_i[19:0]    ----------->     |                            |  ----------->
s_axis_write_desc_valid_i        ----------->     |                            |  ----------->
s_axis_write_data_tdata_i[31:0]  ----------->     |                            |  ----------->
s_axis_write_data_tvalid_i       ----------->     |                            |  ----------->
```

```
s_axis_write_data_tlast_i        ----------->    |                            |  ----------->
m_axi_awready_i                  ----------->    |                            |  ----------->
m_axi_wready_i                   ----------->    |        secure_dma_wr       |  ----------->
m_axi_bresp_i[1:0]               ----------->    |                            |  ----------->
m_axi_bvalid_i                   ----------->    |                            |  ----------->
enable_i                         ----------->    |                            |  ----------->
                                                 |                            |  ----------->
                                                 |                            |  ----------->
                                                 |                            |  ----------->
                                                 +----------------------------+
```

2.1 描述符输入

1. `s_axis_write_desc_addr_i` ：写起始地址。
2. `s_axis_write_desc_len_i` ：写长度（ `bytes-1` 编码）。
3. `s_axis_write_desc_valid_i` ：描述符有效。
4. `s_axis_write_desc_ready_o` ：可接收描述符。

- `len_i` 为 `N-1` ，因此实际字节数为 `N = len_i + 1` 。

2.2 状态输出

1. `m_axis_write_desc_status_error_o[3:0]` ：错误码。
2. `m_axis_write_desc_status_valid_o` ：状态有效脉冲（1 拍）。

错误码定义：

1. `0` ： `DMA_ERROR_NONE`
2. `1` ： `DMA_ERROR_SLVERR`
3. `2` ： `DMA_ERROR_DECERR`
4. `3` ： `DMA_ERROR_RESP` （其他/保底）

2.3 AXIS 写数据输入

1. `s_axis_write_data_tdata_i`
2. `s_axis_write_data_tvalid_i`
3. `s_axis_write_data_tready_o`
4. `s_axis_write_data_tlast_i` （当前版本不参与控制）

2.4 AXI 写主接口

AW 通道： `m_axi_aw*` W 通道： `m_axi_w*` B 通道： `m_axi_b*`

3. 状态机设计

状态：

1. `ST_IDLE`
2. `ST_ISSUE`

3. `ST_W`
4. `ST_B`
5. `ST_DONE`

3.1 ST_IDLE

动作：

1. `desc_ready = enable_i`。
2. 若描述符握手成功：

- 锁存 `addr`。
- 计算总字节 `op_bytes_count = len + 1`。
- 清错误状态。
- 进入 `ST_ISSUE`。

3.2 ST_ISSUE（发 AW）

动作：

1. 若 `AWVALID` 尚未拉起，计算本 burst 传输字节 `tr_bytes_count`。
2. 约束：

- 不超过 `AXI_MAX_BURST_SIZE`。
- 不跨 4KB 边界。

3. 生成并拉起：

- `AWADDR = addr_reg`
- `AWLEN = beats-1`
- `AWVALID = 1`

4. 预更新：

- `addr += tr_bytes_count`
- `op_bytes_count -= tr_bytes_count`
- `burst_beats = AWLEN + 1`
- `burst_last_desc = (op_bytes_count_next == 0)`

转移：`AW` 握手后进入 `ST_W`。

3.3 ST_W（送 WDATA）

动作：

1. 仅在输入 FIFO 非空时送 W。
2. 若当前未拉 `WVALID`，从 FIFO 取队首填 `WDATA` 并拉 `WVALID`。
3. `WLAST` 在 `burst_beats == 1` 时拉高。
4. 每次 `W` 握手后：

- `burst_beats--`
- 同时 FIFO 出队 1 beat

转移：当最后一个 beat 握手（`WLAST` 对应 beat）后进入 `ST_B`。

3.4 ST_B（等写响应）

动作：

1. `BREADY=1`。
2. `B` 握手后检查 `BRESP`：非 OKAY 时记录错误码（仅记录首个错误）。

转移：

1. 若 `burst_last_desc=1`：`status_valid` 打 1 拍，进 `ST_DONE`。
2. 否则回 `ST_ISSUE` 发下一个 burst。

3.5 ST_DONE

动作：清理收尾，下一拍回 `ST_IDLE`。

4. 4KB 边界切分机制

AXI 约束：一次 burst 不能跨 4KB 边界。

实现方式：

1. 先看剩余字节数 `op_bytes_count` 与 `AXI_MAX_BURST_SIZE`。
2. 若本次候选长度会跨 `addr[11:0]` 所在 4KB 边界，则截断到边界末。
3. 取截断后的 `tr_bytes_count` 生成 `AWLEN`。

效果：

1. 任何 burst 都合法。
2. 大消息会自动拆成多个合法 burst。

5. 输入 FIFO 与反压

FIFO 结构：

1. 深度 `2^5 = 32` words。
2. 指针：`in_fifo_wr_ptr_reg` / `in_fifo_rd_ptr_reg`。
3. 半满标志：`in_fifo_half_full_reg`。

反压策略：

1. `s_axis_write_data_tready_o = !in_fifo_half_full_reg`。
2. FIFO 达半满后即对上游施加反压，留出突发缓冲余量。

push/pop 条件：

1. push： `tvalid && tready && !full`
2. pop： `w_hs && !empty`

6. valid/ready 保持策略

模块遵循 AXI 要求：

1. `AWVALID` 一旦拉起，保持到 `AWREADY` 握手。
2. `WVALID` 一旦拉起，保持到 `WREADY` 握手。
3. `BREADY` 仅在 `ST_B` 拉高。

代码中体现：

1. `m_axi_awvalid_next = m_axi_awvalid_reg && !m_axi_awready_i`。
2. `m_axi_wvalid_next = m_axi_wvalid_reg && !m_axi_wready_i`。

7. 错误处理策略

1. `BRESP=OKAY` ：不记错。
2. `BRESP!=OKAY` ：映射为 `SLVERR/DECERR/RESP`。
3. 多个 burst 出错时，只保留首个错误码（便于快速定位首次失败点）。
4. 描述符最终完成时再统一打出一次 `status_valid`。

8. enable 门控

`enable_i` 仅门控“是否接受新描述符”：

1. `desc_ready = enable_i`。
2. 已经启动的事务会继续跑完。

在 secure_combo 中：仅在 AES 路径开启时 `enable_i=1` （因为只有 AES 需要写回密文）。

9. 设计限制与注意事项

1. `s_axis_write_data_tlast_i` 当前未用于收尾判定，长度完全由描述符驱动。
2. 默认 `WSTRB` 恒全 1，不支持部分字节写掩码语义。
3. 单描述符串行处理，无多描述符并发。
4. 若上游供数不足，FSM 会停在 `ST_W` 等待 FIFO 来数。

4.8 时序 Walkthrough

本文回答一个问题： `secure_combo_top` 在一次任务里，信号是按什么时序从 `start` 走到 `done/err` 的？

覆盖模式：

1. AES_ONLY
2. SHA_ONLY

3. AES_SHA（并行）

Phase 0: 配置阶段（CPU/APB）

CPU 写寄存器：

1. `MODE_CFG`（mode、aes_enc_dec、aes_mode、aes_key_len 等）
2. `SRC_BASE / DST_BASE / MSG_LEN`
3. `AES_KEY[] / AES_IV[]`（若 AES 参与）
4. `CTRL.START=1`

`secure_cfg` 产生单拍 `start_pulse`。

Phase 1: start 受理与前置检查

顶层关键条件：

```
start_accept_w = start_pulse && (msg_len != 0) && (!mode_aes || aes_cfg_supported)
```

含义：

1. 消息长度不能为 0。
2. 如果涉及 AES，配置必须受支持：
    - `aes_mode` 当前只支持 ECB/CBC
    - `aes_key_len` 当前只支持 128/256

若 `start_accept_w=1`：

1. `rd_desc_valid_r <= 1`（一定启动读 DMA）
2. `wr_desc_valid_r <= mode_aes`（仅 AES 路需要写回）
3. AES 路置 `aes_init_pulse_r` 并进入调度状态机

`secure_flow_ctrl` 在收到 `start_i` 后进入 busy 过程（`busy=1`）。

Phase 2: DMA 描述符握手

读写 DMA 的描述符 valid 都是“保持型”：

1. `rd_desc_valid_r` 直到 `rd_desc_ready` 才清零
2. `wr_desc_valid_r` 直到 `wr_desc_ready` 才清零

这样避免 ready 晚到时丢启动。

Phase 3: 读通路拉流 + 分流

`secure_dma_rd` 发 AXI AR/R，把源数据转为 AXIS：`rd_tdata/valid/ready/last`。

`secure_axis_split` 根据模式把流广播到两路：

1. AES 路（ `path0_en = mode_aes` ）
2. SHA 路（ `path1_en = mode_sha` ）

每路后接一个 `secure_fifo` （33-bit： `last+data` ）做解耦。

关键点：分流器用联合 ready，确保“同一拍数据”在使能路径一致前进，不错位。

Phase 4A: AES 路执行（若 mode_aes=1）

数据与控制链：

1. `fifo_aes` -> `axis32_to_block128` 聚 128b。
2. 顶层 AES 调度 FSM（ `AES_ST_IDLE/RUN/WAIT` ）采用 one-block-in-flight。
3. 条件满足时打 `aes_next_pulse_r` ，驱动 `aes_core` 处理一个块。
4. 结果 `aes_result_valid` 后，经 `block128_to_axis32` 打散。
5. `secure_dma_wr` 通过 AW/W/B 写回 `dst_base` 。

调度保护：

1. `aes_key_ready_ok_r` 要求先见过 key_ready 拉低再拉高，避免复位后瞬态导致误启动。
2. 只有 `aes_ibuf_valid && !aes_obuf_full` 才发下一块。

Phase 4B: SHA 路执行（若 mode_sha=1）

链路：

1. `fifo_sha` -> `secure_sha_path`
2. `secure_sha_path` 负责块化/padding/驱动 `sha256_core`
3. 输出： `sha_digest_w` 、 `sha_done_w` 、 `sha_err_w`

顶层把 digest 映射为 `hash_regs[0..7]` ，供 APB 读。

Phase 5: 完成条件汇聚

顶层向 `secure_flow_ctrl` 提供完成钩子：

1. `rd_done = rd_stat_valid`
2. `wr_done = mode_aes ? wr_stat_valid : 1`
3. `aes_done = mode_aes ? wr_stat_valid : 1` （当前实现与 wr_done 等价）
4. `sha_done = mode_sha ? sha_done_w : 1`

因此按模式可得：

1. AES_ONLY： `rd_done && wr_done`
2. SHA_ONLY： `rd_done && sha_done`
3. AES_SHA： `rd_done && wr_done && sha_done`

Phase 6: 错误汇聚

`datapath_err` 来源：

1. `rd_stat_err != 0`
2. `wr_stat_err != 0` （仅 AES 模式参与）
3. `sha_err_w`

错误来源编码（ `err_src` ）：

1. `1` ：RD DMA
2. `2` ：WR DMA
3. `3` ：SHA PATH

`secure_flow_ctrl` 接到错误后输出 `err=1` ，并给出 `err_code/err_src` 。

Phase 7: 任务结束与中断

结束信号：

1. 成功： `done=1`
2. 失败： `err=1`

中断： `intr = intr_en & (done | err)`

CPU 侧通常行为：

1. 轮询状态寄存器直到 `done|err` 。
2. 若 `done` ：读取 hash、校验内存结果。
3. 若 `err` ：读取 `err_code/err_src` 定位问题。

4.9 secure_combo 综合报告

1. 范围说明

本报告覆盖 `secure_combo` 相关关键模块在 `TSMC_013 typical.db` 下的综合结果：

1. `sha256_core`
2. `aes_core`
3. `secure_dma_rd`
4. `secure_dma_wr`
5. `secure_sha_path`
6. `secure_combo_top`

综合库：

- `/home/yian/Prj/TSMC_013/TSMC_013/synopsys/typical.db`

时钟约束：

- `10ns` 时钟周期

- `0.2ns` 时钟不确定度

2. 结果总表

模块	Cell Area	Setup Slack	Hold	结论
sha256_core	80578.97	+1.02ns	0	通过
aes_core	237063.97	+4.66ns	0	通过
secure_dma_rd	57302.53	+6.06ns	0	通过
secure_dma_wr	53103.16	+5.97ns	0	通过
secure_sha_path	145479.06	+0.47ns	0	通过
secure_combo_top	643559.02	+0.22ns	0	通过

说明：

1. `secure_dma_rd` 这里采用的是清理 declaration initialization 之后的最新结果。
2. `secure_combo_top` 采用的是带最新 `secure_dma_rd` 的重跑结果。
3. `aes_core` 当前结果采用加入 `set_fix_hold` 后的重跑版本，原先独立模块的 2 条轻微 hold 已收口。

3. 各模块结果与报告路径

3.1 sha256_core

报告：

- `out/dc_tsmc013/sha256_core_qor.rpt`
- `out/dc_tsmc013/sha256_core_area.rpt`
- `out/dc_tsmc013/sha256_core_timing.rpt`
- `out/dc_tsmc013/sha256_core_check.rpt`

结论：

1. 面积适中。
2. 时序满足 `10ns`。
3. 无 setup/hold 违例。

3.2 aes_core

报告：

- `out/dc_tsmc013/aes_core_fixhold_qor.rpt`
- `out/dc_tsmc013/aes_core_fixhold_area.rpt`
- `out/dc_tsmc013/aes_core_fixhold_hold.rpt`
- `out/dc_tsmc013/aes_core_check.rpt`

结论：

1. 面积是几个核心算法模块里最大的。

2. 加入 `set_fix_hold [get_clocks clk_i]` 并重跑后，setup/hold 都满足当前约束。
3. 该问题本质上是若干很短的控制反馈路径，不是 AES 主数据通路过慢。
4. 在当前前端综合约束下，`aes_core` 已可视为 clean。

3.3 secure_dma_rd

报告：

- `out/dc_tsmc013/secure_dma_rd_qor_clean.rpt`
- `out/dc_tsmc013/secure_dma_rd_area_clean.rpt`
- `out/dc_tsmc013/secure_dma_rd_timing_clean.rpt`
- `out/dc_tsmc013/secure_dma_rd_check_clean.rpt`

结论：

1. 清理 declaration initialization 后，`VER-708` 告警消失。
2. 同时把内部固定常量从 `parameter` 改为 `localparam`，`VER-329` 也消失。
3. 该模块 setup/hold 都较宽松。
4. 现存 `check_design` 关注点主要是接口语义，不是时序或可综合性问题。

3.4 secure_dma_wr

报告：

- `out/dc_tsmc013/secure_dma_wr_qor.rpt`
- `out/dc_tsmc013/secure_dma_wr_area.rpt`
- `out/dc_tsmc013/secure_dma_wr_timing.rpt`
- `out/dc_tsmc013/secure_dma_wr_check.rpt`

结论：

1. setup/hold 都通过。
2. 当前 `s_axis_write_data_tlast_i` 未参与功能判定，综合检查里会看到未连接提示。
3. 这是“按长度收尾而非依赖 `tlast`”的当前架构选择。
4. 当前 RTL 已统一采用 `localparam + reset explicit init` 风格，没有 `VER-708/VER-329` 这类低价值综合告警。

3.5 secure_sha_path

报告：

- `out/dc_tsmc013/secure_sha_path_qor.rpt`
- `out/dc_tsmc013/secure_sha_path_area.rpt`
- `out/dc_tsmc013/secure_sha_path_timing.rpt`
- `out/dc_tsmc013/secure_sha_path_check.rpt`

结论：

1. 这是本次重构中最关键的模块之一。
2. 旧版“整条消息缓存”不适合高效综合；新版“流式分块 + 64B block buffer”已可稳定综合。
3. 当前关键路径主要位于长度/padding/block-buffer 控制逻辑。
4. `sha256_core` 本体约占该模块面积的一半以上，符合“核 + 外围路径控制”的结构特征。

3.6 secure_combo_top

报告：

- `out/dc_tsmc013/secure_combo_top_qor.rpt`
- `out/dc_tsmc013/secure_combo_top_area.rpt`
- `out/dc_tsmc013/secure_combo_top_timing.rpt`
- `out/dc_tsmc013/secure_combo_top_check.rpt`
- `out/dc_tsmc013/secure_combo_top_constraints.rpt`

结论：

1. 顶层已完整综合通过。
2. setup/hold 均满足当前约束。
3. 最差路径仍在 `secure_sha_path` 外围控制逻辑。
4. 从系统视角看，当前 `secure_combo` 已达到“真实工艺库可综合”的状态。

4. 本次综合相关代码改动

4.1 secure_dma_rd

文件：

- `rtl/ip/secure_combo/secure_dma_rd.v`

本次为降低综合噪声与改善 RTL 风格，做了两类改动：

1. 删除 declaration initialization

- 原先大量 `reg xxx = init` 的写法已去掉。
- 改为：
  - 声明不带初值
  - reset 分支显式初始化
  - 组合块显式给默认值

2. 内部固定常量改为 `localparam`

- 避免 DC 对“内部常量仍写成 parameter”的风格告警。

这些改动不改变模块协议语义，只让 RTL 更贴近真实硬件复位行为。

4.2 secure_sha_path

文件：

- `rtl/ip/secure_combo/secure_sha_path.v`

本次不是小修，而是架构重构：

1. 从“整条消息缓存”改成“流式分块处理”
2. 只保留一个 `64-byte block buffer`
3. 在本模块内完成标准 SHA padding
4. 用 `core_word_q` 稳定送 word 给 `sha256_core`

这项改动直接把 `secure_sha_path` 从“不适合顶层综合”的状态，变成了“可稳定通过真实工艺库综合”的状态。

5. 告警分类整理

5.1 已解决的综合告警

1. `VER-708`

- 来源：`secure_dma_rd` 中 declaration initialization
- 处理：改成纯 reset 风格
- 当前状态：已消失

2. `VER-329`

- 来源：`secure_dma_rd` 内部固定常量写成 `parameter`
- 处理：改为 `localparam`
- 当前状态：已消失

5.2 当前保留但可接受的告警

1. 高扇出告警

- 例如 `secure_combo_top` 中 `u_secure_sha_path/u_sha256_core/clk_i`
- 属于综合阶段常见现象，不是功能错误

2. 未连接端口

- `m_axi_rlast_i`
- `s_axis_write_data_tlast_i`
- `s_axis_last_i`
- `apb_paddr_i[11:8]`

这些都来自当前接口/架构选择，而不是 RTL 漏接导致的功能错误。

3. feedthrough / shorted outputs / constant outputs

- 主要来自：

    - `secure_axis_split` 的直通复制结构
    - AXI 固定 `size/burst`
    - `apb_pready_o = 1`
- 这类提示是结构特征，不代表设计不可用

4. `max_leakage_power` 违例

- 这是因为脚本未设置实际 leakage 目标，默认显示 `0`
- 不代表时序或逻辑综合失败

5.3 当前仍建议后续关注的点

1. `secure_sha_path` 仍是顶层关键路径来源
2. 若后续做更正式综合，需要加入更完整的 IO/false path/multicycle 等约束

6. 工程层面的结论

当前 `secure_combo` 工程已经达到以下状态：

1. 关键子模块可在真实工艺库下综合
2. 顶层 `secure_combo_top` 可完整综合
3. 主要综合阻塞点已被解决
4. 当前剩余告警大部分属于结构性提示，而不是致命问题

从数字 IC 工程角度，这意味着：

- 当前设计已经从“仿真可跑”推进到了“真实工艺库可综合”的阶段

7. 后续建议

1. 若继续推进到更正式的综合交付：

- 增加更完整的 IO 约束
- 单独处理 `aes_core` 的轻微 hold
- 对高扇出时钟/复位做后端视角评估

2. 若继续推进到 SoC 级：

- 进一步综合 `fabric` 与 `soc_top`
- 再做一次系统级面积/时序归档

3. 若继续做代码清理：

- 持续减少“接口保留但当前未使用”的端口
- 或在文档中更明确地把这些端口标为“保留/未使用”